



deeplearning.ai

# Introduction to Deep Learning

---

# Welcome



- AI is the new Electricity
- Electricity had once transformed countless industries: transportation, manufacturing, healthcare, communications, and more
- AI will now bring about an equally big transformation.

# What you'll learn



Courses in this sequence (Specialization):

1. Neural Networks and Deep Learning
2. Improving Deep Neural Networks: Hyperparameter tuning, Regularization and Optimization
3. Structuring your Machine Learning project
4. Convolutional Neural Networks *CNN*
5. Natural Language Processing: Building sequence models  
*RNN, LSTM*

*train / dev / test*  
*end-to-end*



deeplearning.ai

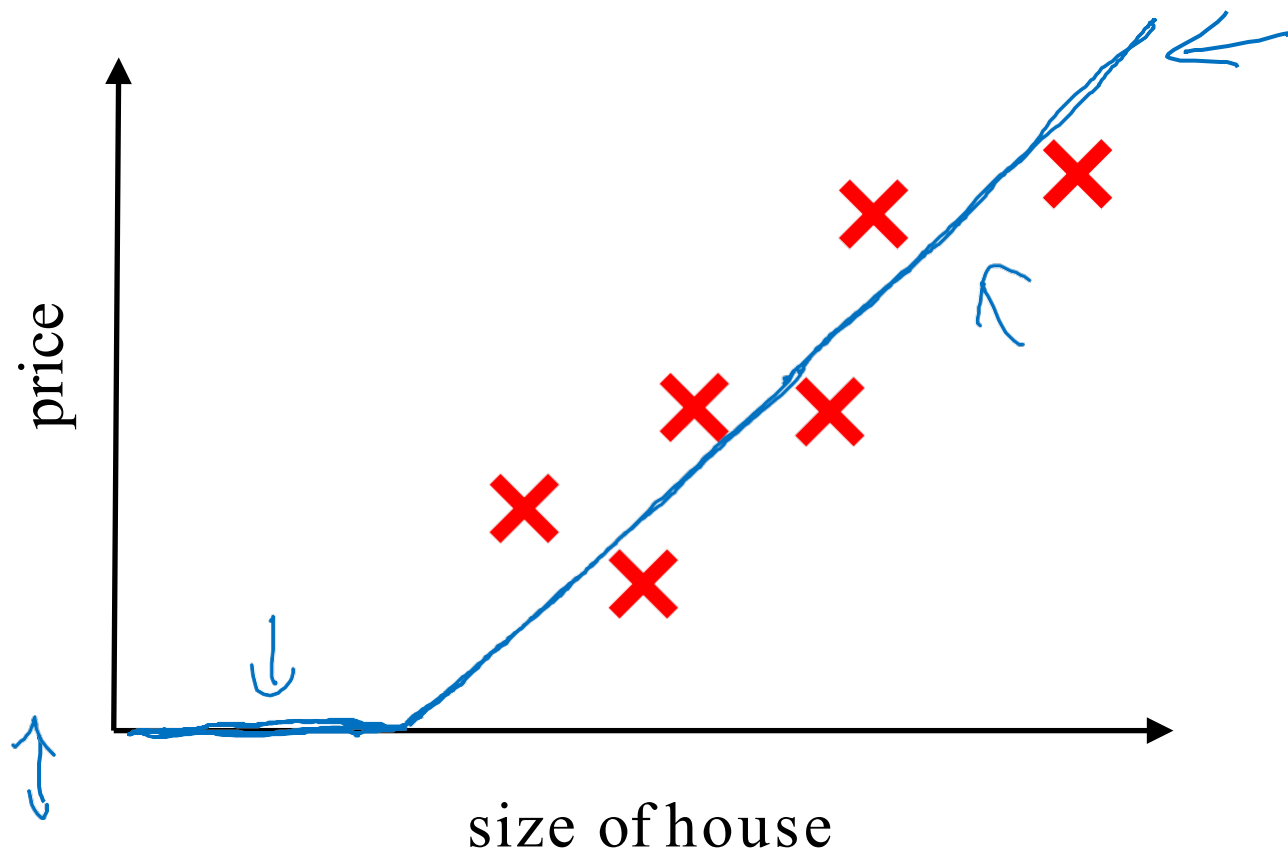
# Introduction to Deep Learning

---

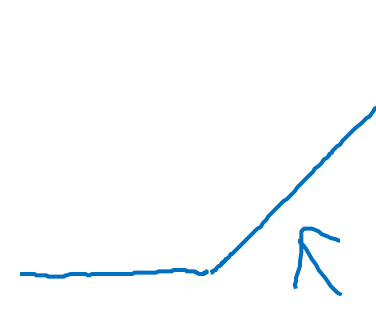
## What is a Neural Network?



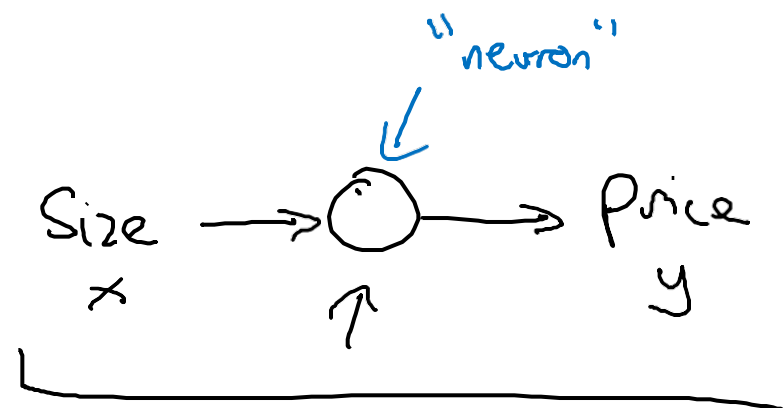
# Housing Price Prediction



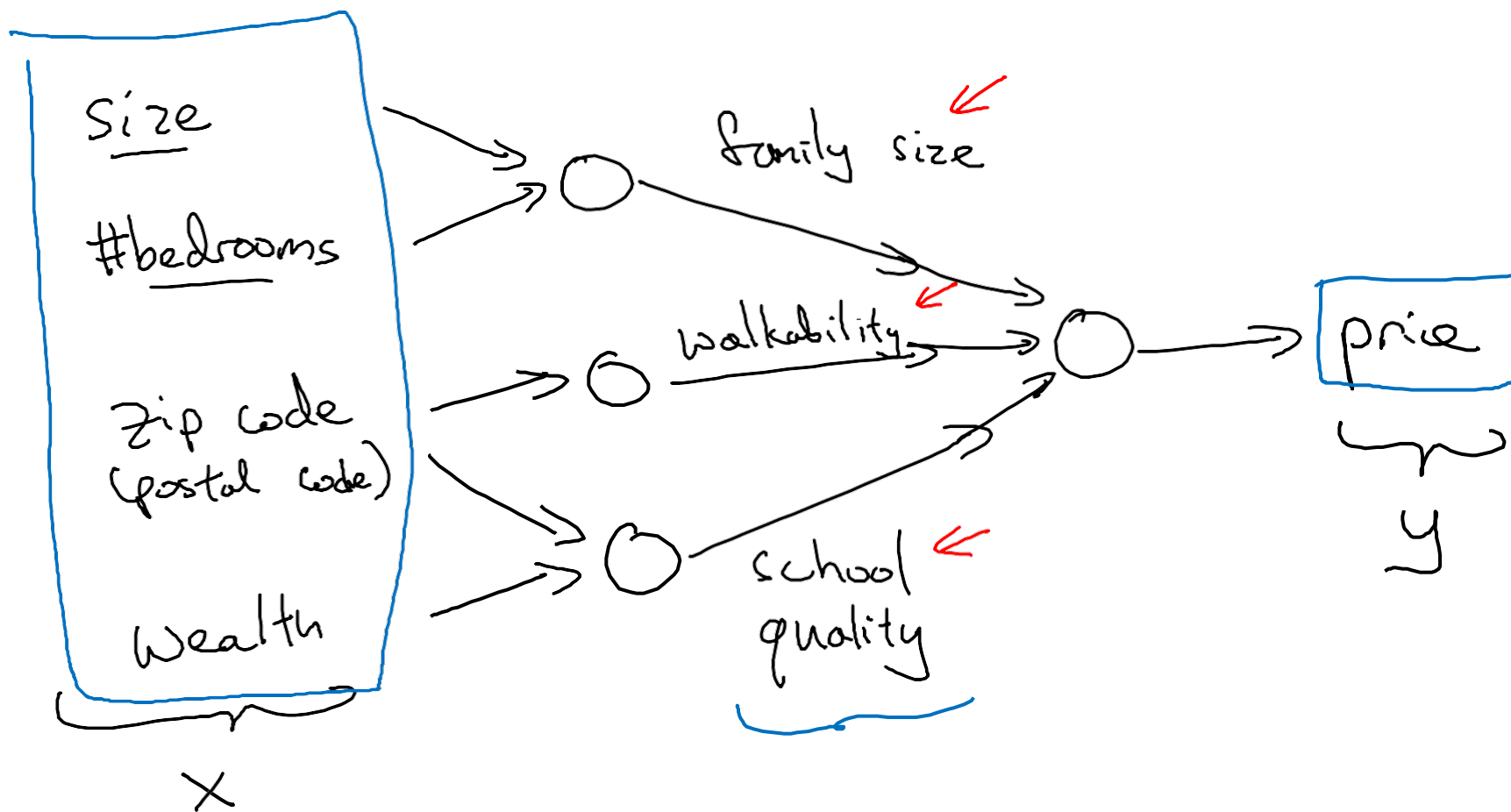
ReLU  
Rectified  
Linear  
Unit



A small graph of the ReLU function, showing a horizontal line at zero for negative inputs and a diagonal line with a slope of one for positive inputs. A blue arrow points to the diagonal segment.

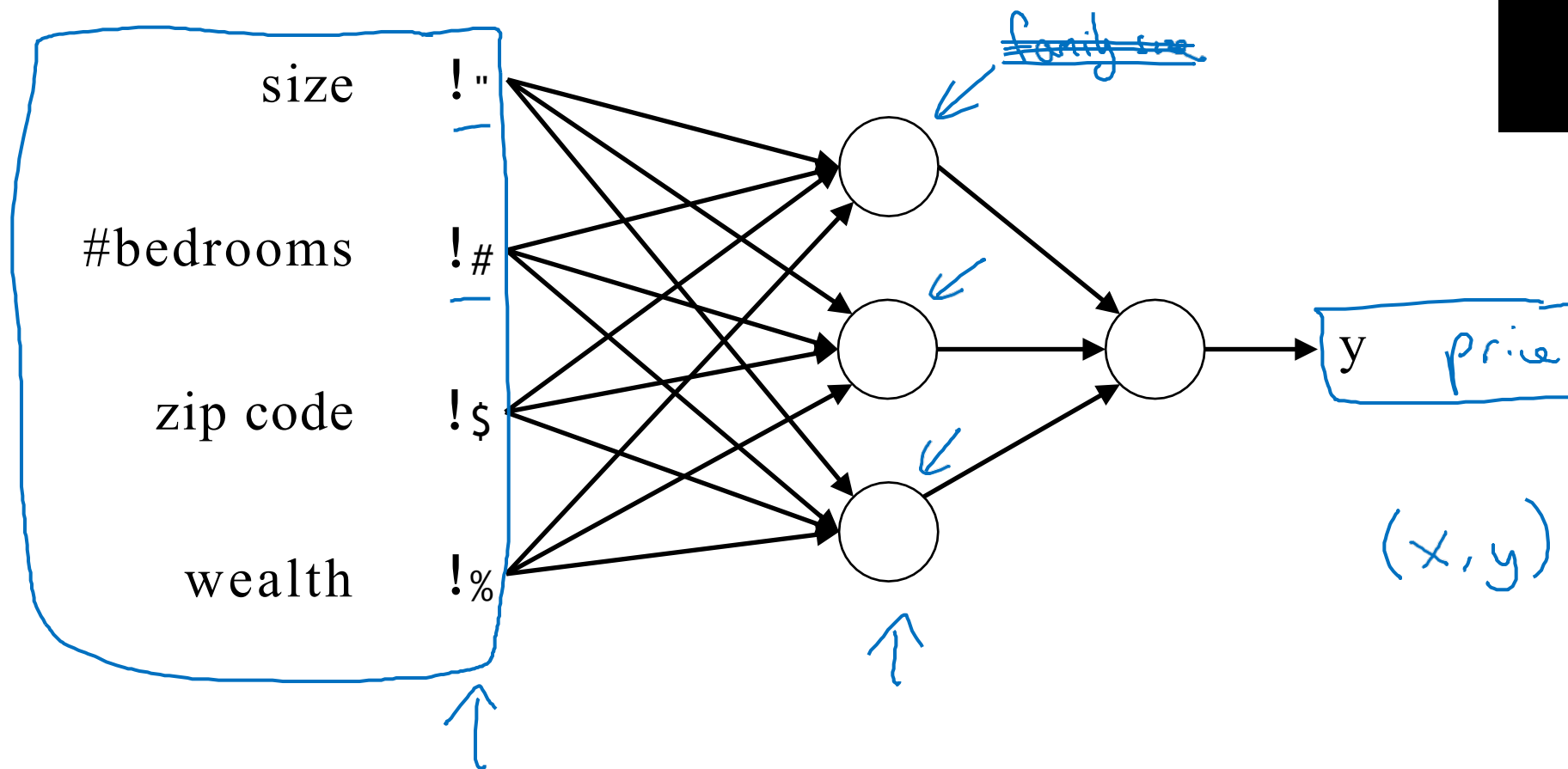


# Housing Price Prediction



# Housing Price Prediction

**Drawing of  
previous Image**





deeplearning.ai

# Introduction to Deep Learning

---

## Supervised Learning with Neural Networks

# Supervised Learning

| Input(x)                               | Output (y)                   | Application         |
|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------|
| Home features                                                                                                           | Price                                                                                                           | Real Estate         |
| Ad, user info                          | Click on ad? (0/1)                                                                                              | Online Advertising  |
| Image                                                                                                                   | Object (1,...,1000)                                                                                             | Photo tagging       |
| Audio                                                                                                                   | Text transcript                                                                                                 | Speech recognition  |
| <u>English</u>                                                                                                          | Chinese                                                                                                         | Machine translation |
| <u>Image</u> , <u>Radar info</u><br> | Position of other cars<br> | Autonomous driving  |

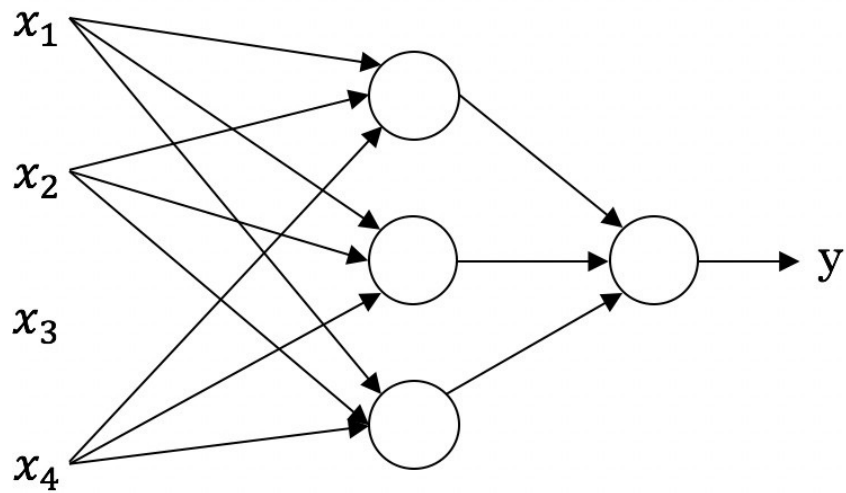
} Standard NN

} CNN

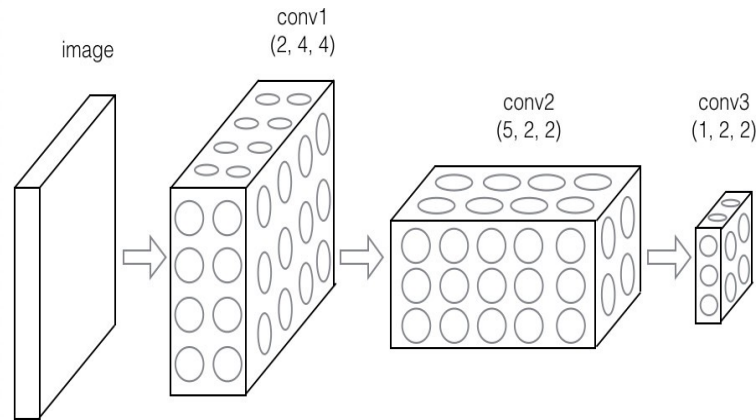
} RNN

} Custom/ Hybrid

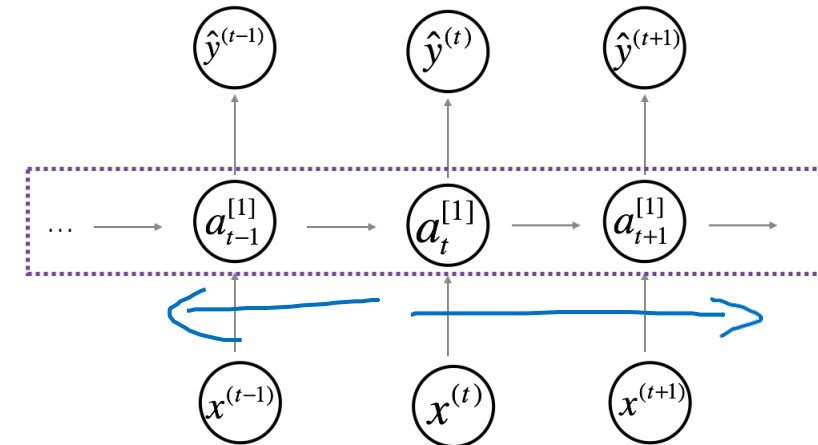
# Neural Network examples



Standard NN



Convolutional NN



Recurrent NN

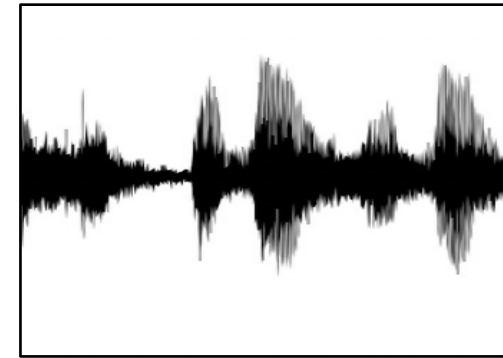
# Supervised Learning

## Structured Data

| Size | #bedrooms | ... | Price (1000\$) |
|------|-----------|-----|----------------|
| 2104 | 3         |     | 400            |
| 1600 | 3         |     | 330            |
| 2400 | 3         |     | 369            |
| ...  | ...       |     | ...            |
| 3000 | 4         |     | 540            |

| User Age | Ad Id | ... | Click |
|----------|-------|-----|-------|
| 41       | 93242 |     | 1     |
| 80       | 93287 |     | 0     |
| 18       | 87312 |     | 1     |
| ...      | ...   |     | ...   |
| 27       | 71244 |     | 1     |

## Unstructured Data



Audio



Image

Four scores and seven  
years ago...

Text



deeplearning.ai

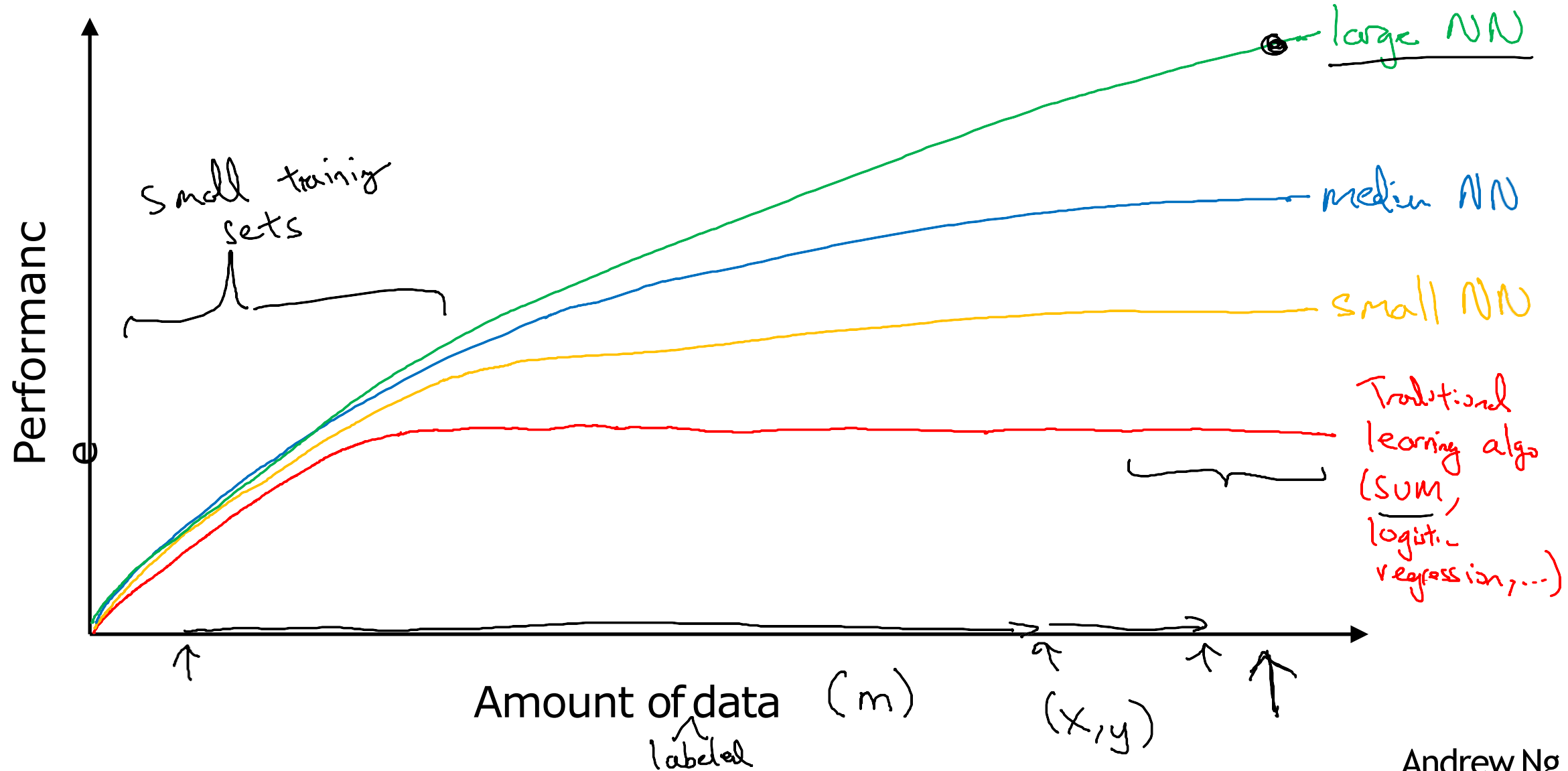
# Introduction to Neural Networks

---

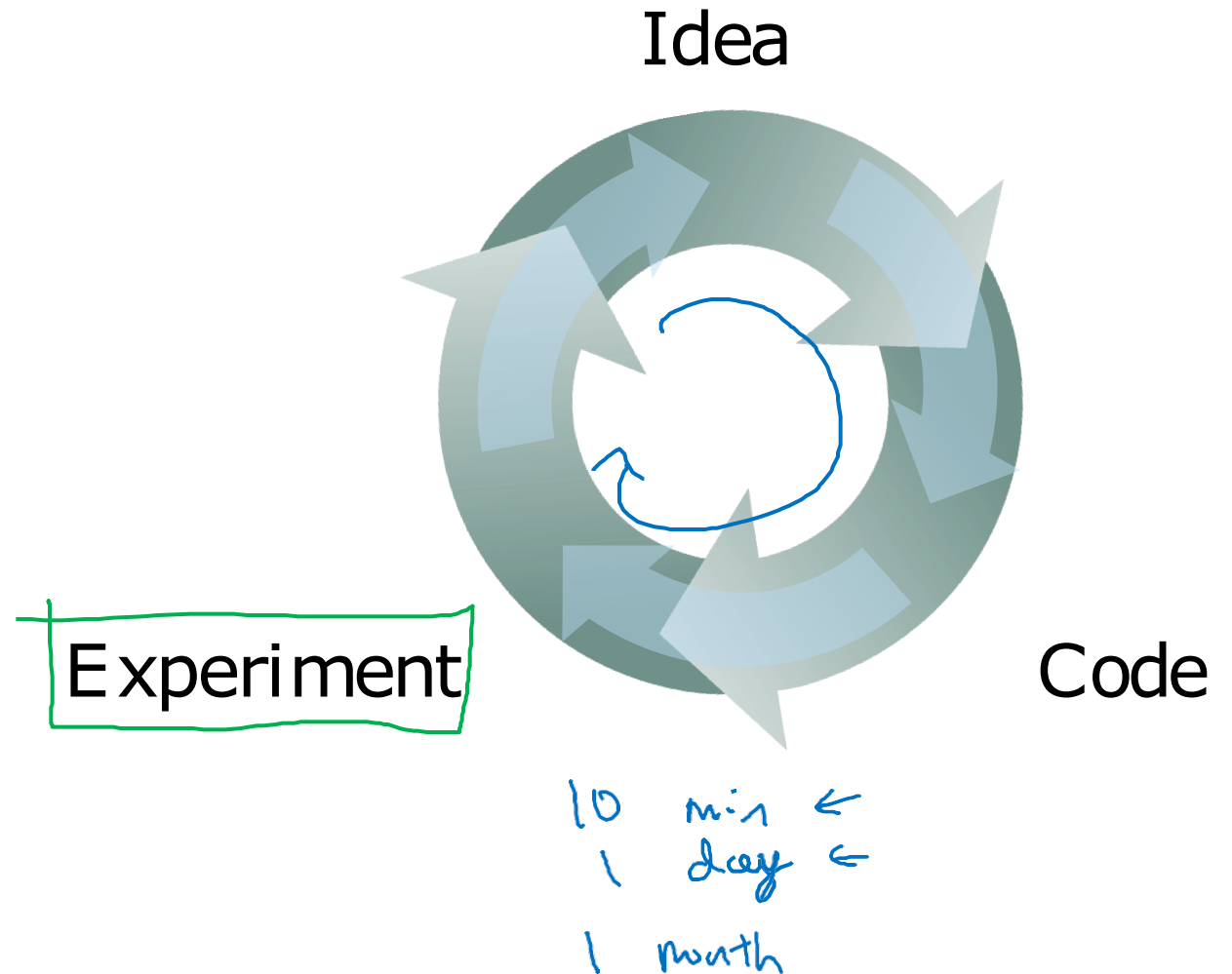
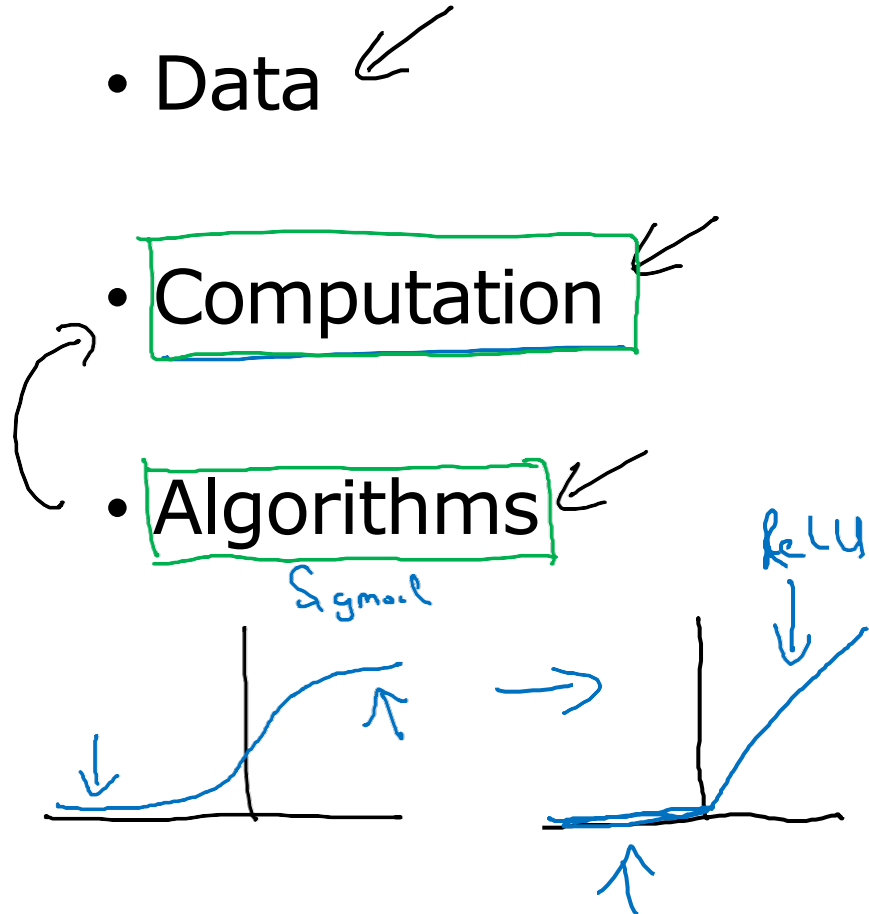
## Why is Deep Learning taking off?



# Scale drives deep learning progress



# Scale drives deep learning progress





deeplearning.ai

# Introduction to Neural Networks

---

## About this Course

# Courses in this Specialization

1. Neural Networks and Deep Learning 
2. Improving Deep Neural Networks: Hyperparameter tuning, Regularization and Optimization
3. Structuring your Machine Learning project
4. Convolutional Neural Networks
5. Natural Language Processing: Building sequence models

# Outline of this Course

Week 1: Introduction

Week 2: Basics of Neural Network programming

Week 3: One hidden layer Neural Networks

Week 4: Deep Neural Networks



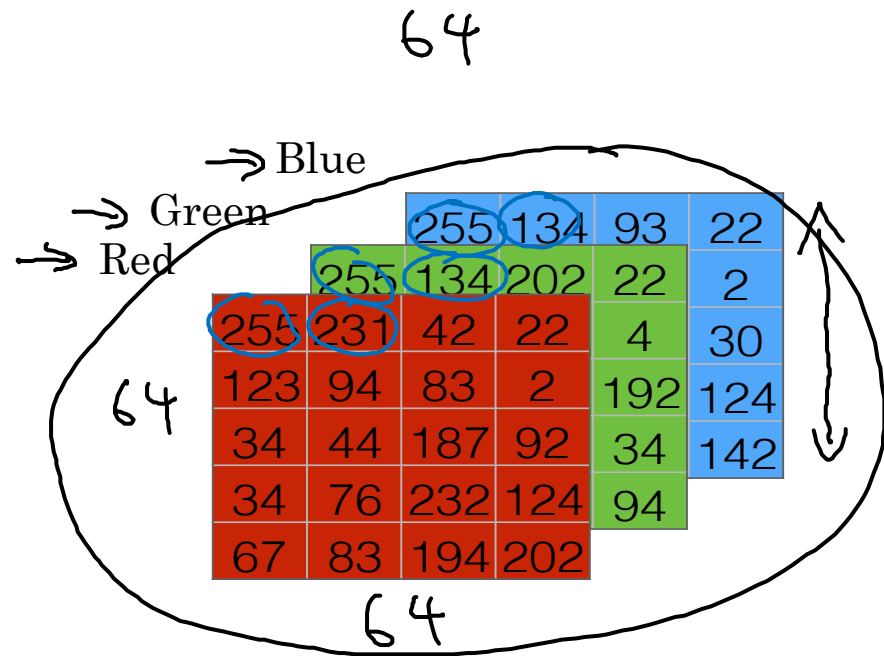
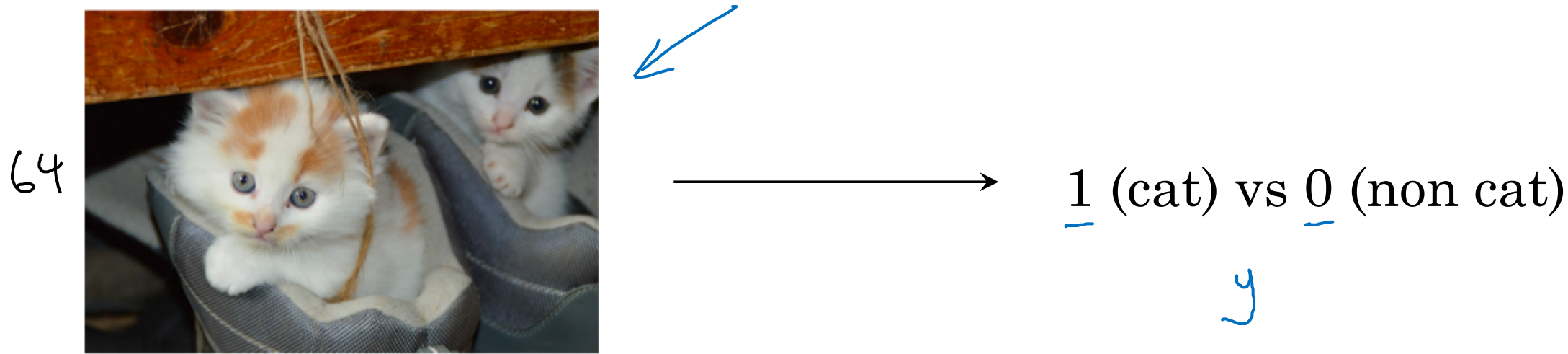
deeplearning.ai

# Basics of Neural Network Programming

---

## Binary Classification

# Binary Classification



$X = \begin{bmatrix} 255 \\ 231 \\ \vdots \\ 255 \\ 134 \\ \vdots \end{bmatrix}$

$64 \times 64 \times 3 = 12288$

$n = n_x = 12288$

$X \longrightarrow y$

# Notation

$$(x, y) \quad x \in \mathbb{R}^{n_x}, y \in \{0, 1\}$$

$$m \text{ training examples: } \{(\underline{x}^{(1)}, \underline{y}^{(1)}), (\underline{x}^{(2)}, \underline{y}^{(2)}), \dots, (\underline{x}^{(m)}, \underline{y}^{(m)})\}$$

$$M = M_{\text{train}}$$

$$M_{\text{test}} = \# \text{test examples.}$$

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

Diagram illustrating the matrix  $X$  with dimensions  $n_x$  (vertical) and  $m$  (horizontal). A crossed-out square diagram shows the structure of the input features  $x^{(1)}, \dots, x^{(m)}$ .

$$X \in \mathbb{R}^{n_x \times m}$$

$$X.\text{shape} = (n_x, m)$$

$$Y = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}]$$

$$Y \in \mathbb{R}^{1 \times m}$$

$$Y.\text{shape} = (1, m)$$





deeplearning.ai

# Basics of Neural Network Programming

---

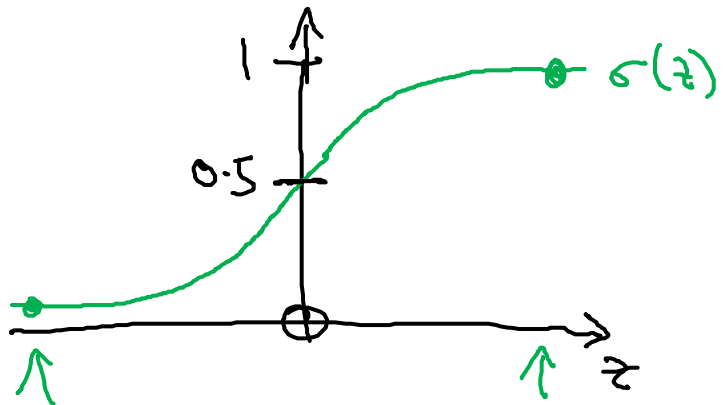
## Logistic Regression

# Logistic Regression

Given  $x$ , want  $\hat{y} = \frac{P(y=1|x)}{0 \leq \hat{y} \leq 1}$   
 $x \in \mathbb{R}^{n_x}$

Parameters:  $\underline{w} \in \mathbb{R}^{n_x}$ ,  $\underline{b} \in \mathbb{R}$ .

Output  $\hat{y} = \sigma(\underbrace{w^T x + b}_z)$



$$x_0 = 1, \quad x \in \mathbb{R}^{n_x+1}$$
$$\hat{y} = \sigma(\theta^T x)$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_{n_x} \end{bmatrix} \quad \left. \begin{array}{l} \} b \leftarrow \\ \} w \leftarrow \end{array} \right\}$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{If } z \text{ large } \sigma(z) \approx \frac{1}{1+0} = 1$$

If  $z$  large negative number

$$\sigma(z) = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + \text{Big num}} \approx 0$$



deeplearning.ai

# Basics of Neural Network Programming

---

## Logistic Regression cost function

# Logistic Regression cost function

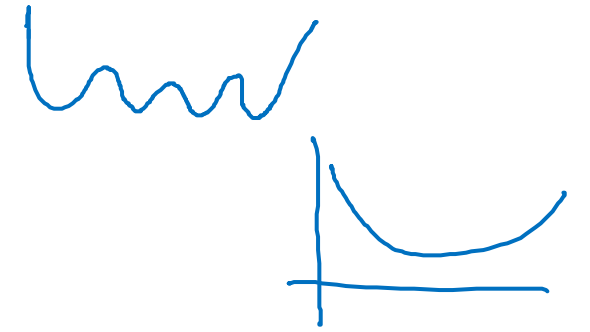
$$\rightarrow \hat{y}^{(i)} = \sigma(w^T \underline{x}^{(i)} + b), \text{ where } \sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}} \quad z^{(i)} = w^T x^{(i)} + b$$

Given  $\{(\underline{x}^{(1)}, y^{(1)}), \dots, (\underline{x}^{(m)}, y^{(m)})\}$ , want  $\hat{y}^{(i)} \approx \underline{y}^{(i)}$ .

$x^{(i)}$   
 $y^{(i)}$   
 $z^{(i)}$   $i$ -th example.

Loss (error) function:

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2$$



$$\mathcal{L}(\hat{y}, y) = - (y \log \hat{y}) + (1-y) \log(1-\hat{y}) \leftarrow$$

If  $y=1$ :  $\mathcal{L}(\hat{y}, y) = -\log \hat{y} \leftarrow$  Want  $\log \hat{y}$  large, want  $\hat{y}$  large.

If  $y=0$ :  $\mathcal{L}(\hat{y}, y) = -\log(1-\hat{y}) \leftarrow$  Want  $\log(1-\hat{y})$  large ... want  $\hat{y}$  small

Cost function:  $J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)})]$



deeplearning.ai

# Basics of Neural Network Programming

---

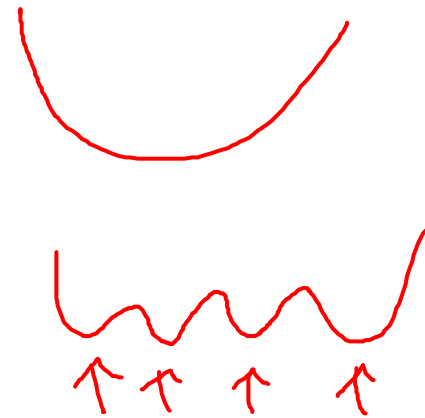
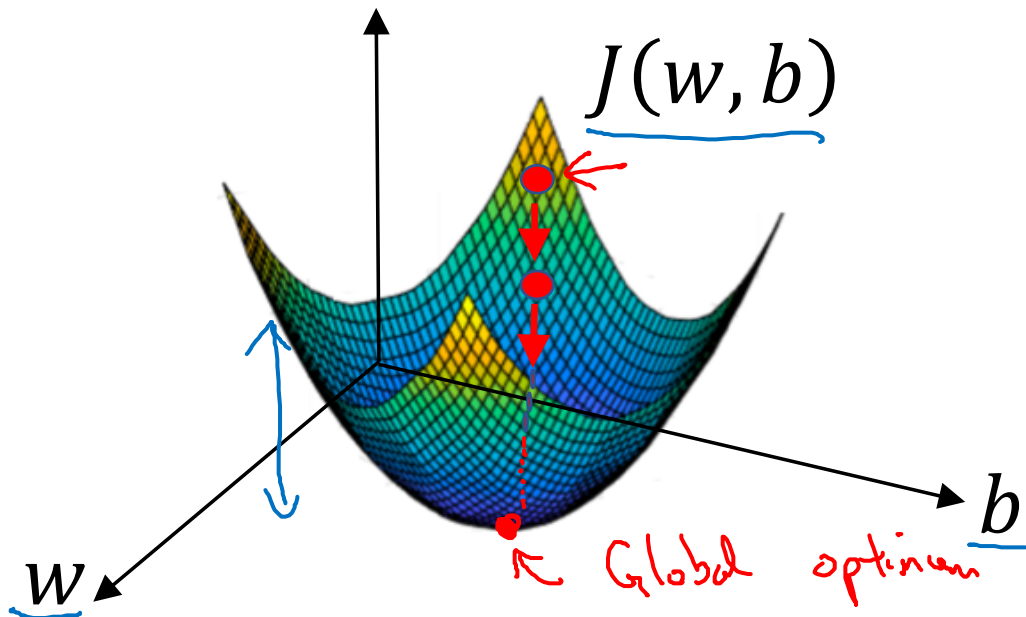
## Gradient Descent

# Gradient Descent

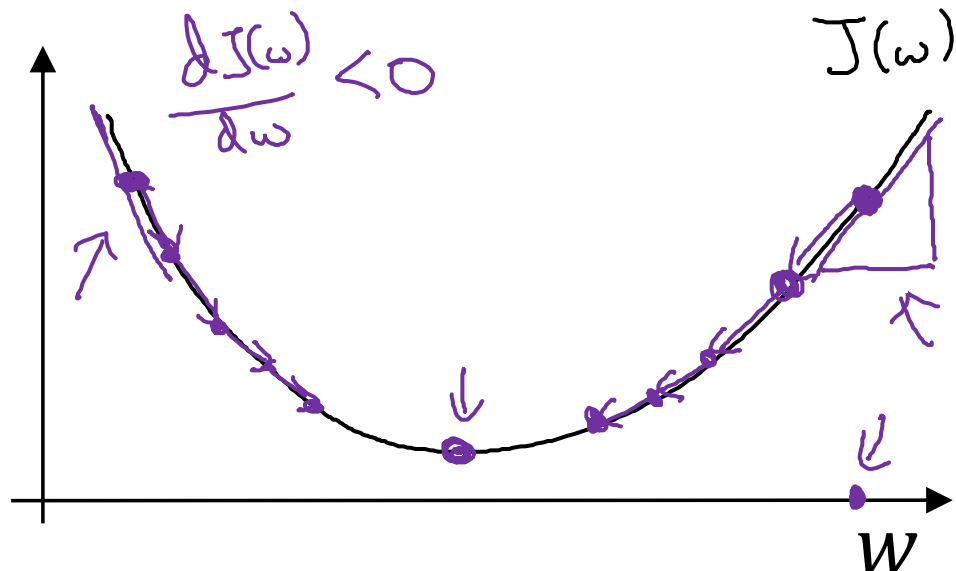
Recap:  $\hat{y} = \sigma(w^T x + b)$ ,  $\sigma(z) = \frac{1}{1+e^{-z}}$   $\leftarrow$

$$\underline{J(w, b)} = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\underline{\hat{y}^{(i)}} , \underline{y^{(i)}}) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Want to find  $w, b$  that minimize  $J(w, b)$



# Gradient Descent



Repeat {

$$w := w - \alpha \frac{dJ(w)}{dw}$$

learning rate

}

$$w := w - \alpha \underbrace{\frac{dJ(w)}{dw}}_{\text{"dw"}}$$

$$\frac{dJ(w)}{dw} = ?$$

$$J(w, b)$$

$$w := w - \alpha \frac{\partial J(w, b)}{\partial w}$$

$$b := b - \alpha \frac{\partial J(w, b)}{\partial b}$$

$$\frac{\partial J(w, b)}{\partial w}$$

$$\frac{\partial J(w, b)}{\partial b}$$

$$\partial$$

$$\partial$$

"partial derivative"  
J

dw

db



deeplearning.ai

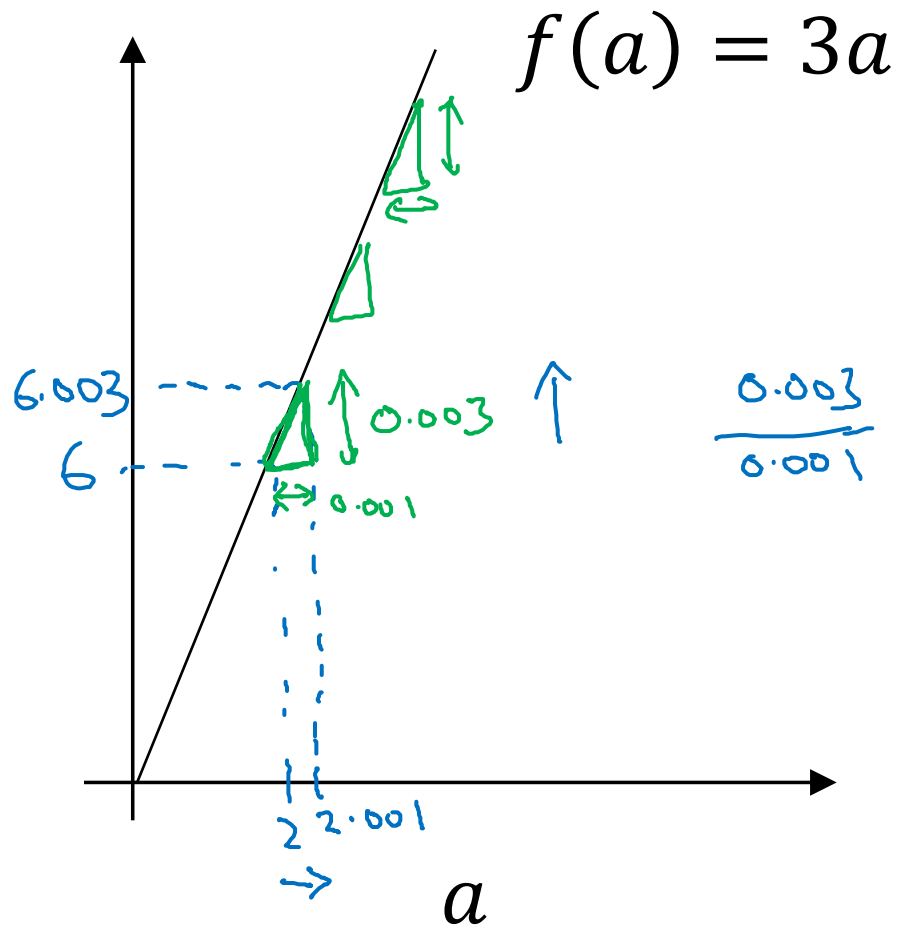
# Basics of Neural Network Programming

---

## Derivatives



# Intuition about derivatives



$$\frac{0.003}{0.001} \quad \frac{\text{height}}{\text{width}}$$

$\rightarrow a = 2 \quad f(a) = 6$   
 $a = 2.001 \quad f(a) = 6.003$

slope (derivative) of  $f(a)$  at  $a = 2$  is 3

$\rightarrow a = 5 \quad f(a) = 15$   
 $a = 5.001 \quad f(a) = 15.003$   
 slope at  $a = 5$  is also 3

$$\frac{df(a)}{da} = 3 = \lim_{\Delta a \rightarrow 0} \frac{f(a + \Delta a) - f(a)}{\Delta a}$$

$\downarrow$   
 $\uparrow$

$0.001 \leftarrow$   
 $0.000000001$   
 $0.000000000001$



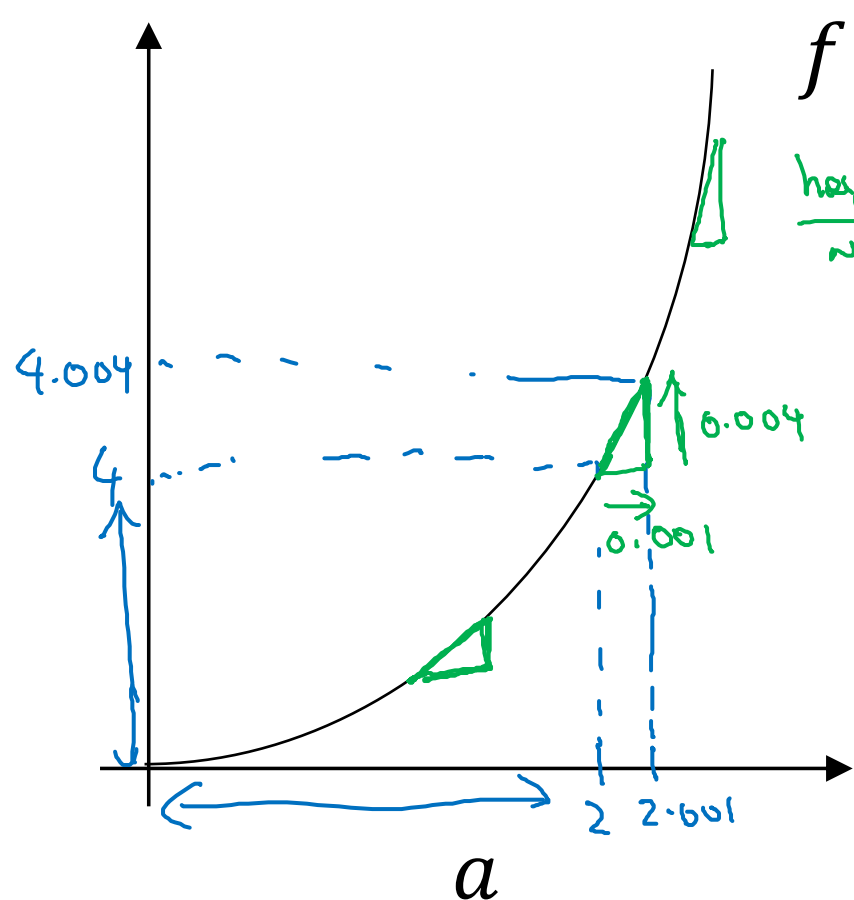
deeplearning.ai

# Basics of Neural Network Programming

---

More derivatives  
examples

# Intuition about derivatives



$$f(a) = a^2$$

height  
width

$$\frac{d}{da} a^2 = 2a$$

$$0.001$$

$$(2a) \times 0.001$$

0.001 ←  
0.000000...01 ←

$a = 2$        $f(a) = 4$   
 $a = 2.001$        $f(a) \approx 4.004$   
 (4.004001) ←  
 slope (derivative) of  $f(a)$  at  
 $a = 2$  is 4.

$$\frac{d}{da} f(a) = 4 \quad \text{when } a = 2.$$

$a = 5$        $f(a) = 25$   
 $a = 5.001$        $f(a) \approx 25.010$

$$\frac{d}{da} f(a) = 10 \quad \text{when } a = 5$$

$$\frac{d}{da} f(a) = \frac{d}{da} a^2 = 2a$$

# More derivative examples

$$f(a) = a^2$$

$$\frac{d}{da} f(a) = \frac{2a}{4}$$

$$a = 2$$

$$f(a) = 4$$

$$a = 2.001$$

$$f(a) \approx 4.004$$

$$f(a) = a^3$$

$$\frac{d}{da} f(a) = \frac{3a^2}{3 \times 2^2 = 12}$$

$$a = 2$$

$$f(a) = 8$$

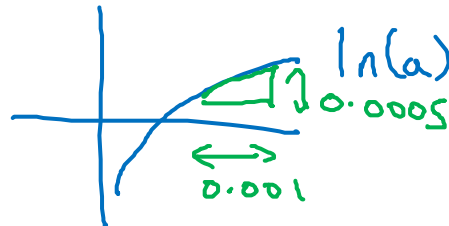
$$a = \underline{2.001}$$

$$f(a) \approx \underline{8.012}$$

$$f(a) = \log_e(a)$$
  

$$\ln(a)$$

$$\frac{d}{da} f(a) = \frac{1}{a}$$



$$\frac{d}{da} f(a) = \boxed{\frac{1}{2}}$$

$$\downarrow a = 2$$

$$\downarrow f(a) \approx 0.69315$$

$$a = \underline{2.001}$$

$$\downarrow \underline{f(a) \approx 0.69365}$$

$$\downarrow 0.0005$$

$$\swarrow \underline{0.0005}$$



deeplearning.ai

# Basics of Neural Network Programming

---

## Computation Graph

# Computation Graph

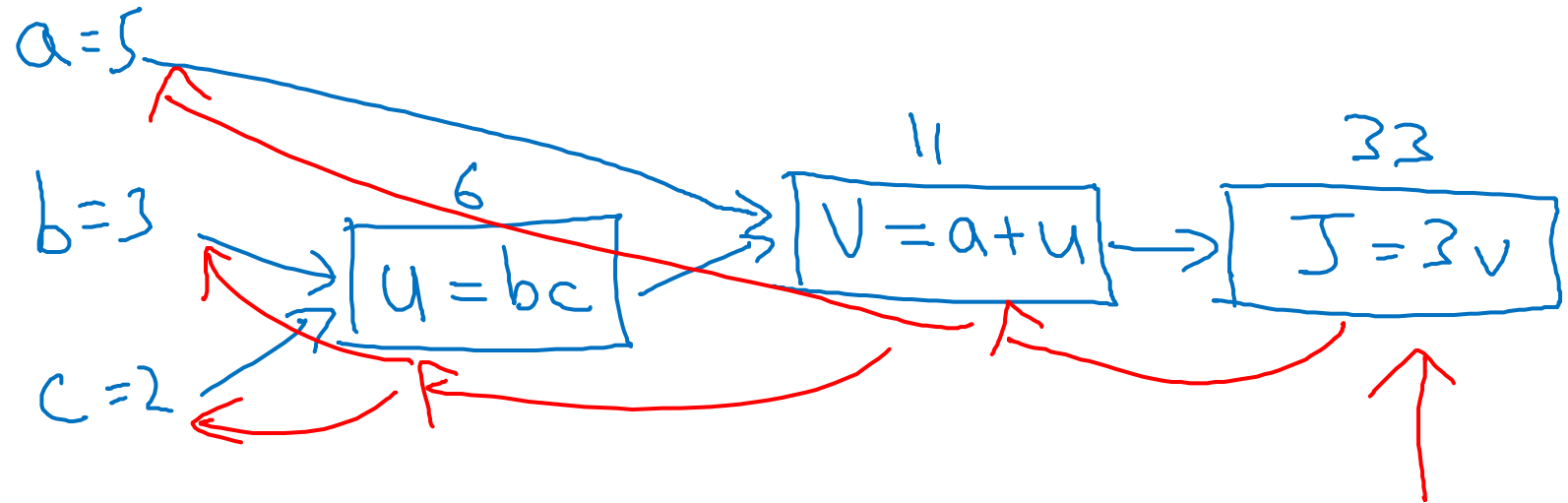
$$J(a,b,c) = 3(a + \underbrace{bc}_u) = 3(5 + 3 \times 2) = 33$$

$$\underbrace{\quad \quad \quad}_J$$

$$u = bc$$

$$V = a + u$$

$$J = 3V$$





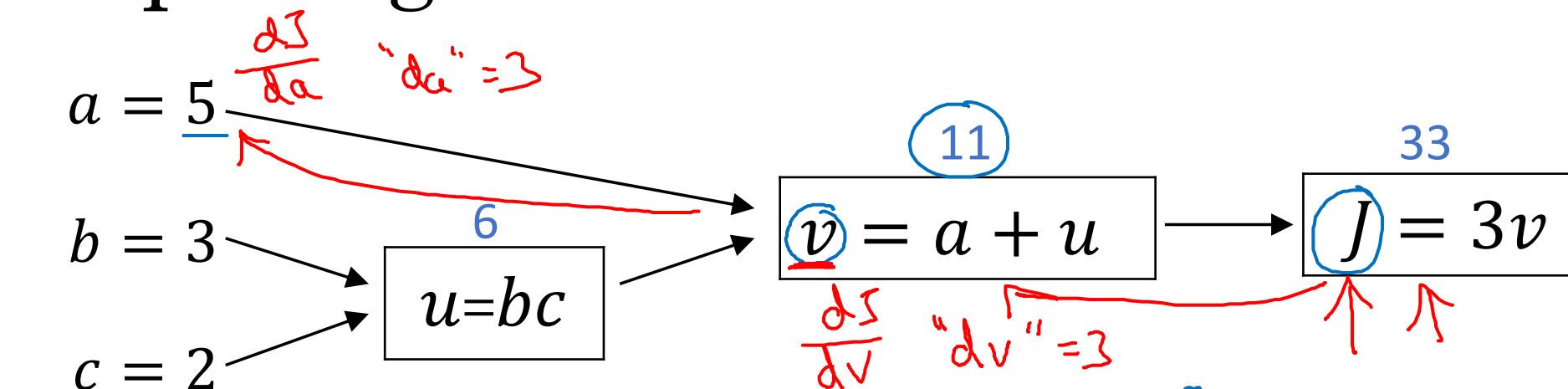
deeplearning.ai

# Basics of Neural Network Programming

---

## Derivatives with a Computation Graph

# Computing derivatives



$$\frac{dJ}{dv} = ? = 3$$

$$\frac{dJ}{da} = 3 = \frac{dJ}{dv} \frac{dv}{da}$$

$$\frac{dv}{da} = 1$$

Handwritten notes in green boxes:

- $\frac{dJ}{dv} = ? = 3$
- $\frac{dv}{da} = 1$

$$a \rightarrow v \rightarrow J$$

$$J = 3v$$

$$v = 11 \rightarrow 11.001$$

$$J = 33 \rightarrow 33.003$$

$$a = 5 \rightarrow 5.001$$

$$\rightarrow v = 11 \rightarrow 11.001$$

$$J = 33 \rightarrow 33.003$$

$$\frac{d \text{ Final Output Var}}{d \text{ var}}$$

$$\frac{dJ}{d \text{ var}}$$

Handwritten note: "dvar"

$$f(a) = 3a$$

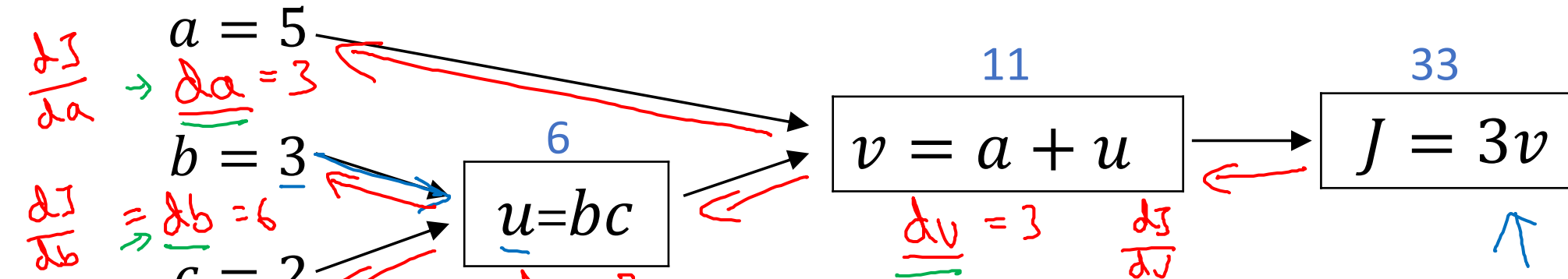
$$\frac{df(a)}{da} = \frac{df}{da} = 3$$

$$J = 3v$$

$$\frac{dJ}{dv} = 3$$



# Computing derivatives



$$\frac{dJ}{du} = 3 = \frac{dJ}{dv} \cdot \frac{dv}{du}$$

$\underbrace{\quad}_{3} \quad \underbrace{\quad}_{1}$

$$\frac{dJ}{db} = \frac{dJ}{du} \cdot \frac{du}{db} = 6$$

$\underbrace{\quad}_{\rightarrow 3} \quad \underbrace{\quad}_{=2}$

$$\frac{dJ}{dc} = \frac{dJ}{du} \cdot \frac{du}{dc} = 9$$

$\underbrace{\quad}_{\rightarrow 3} \quad \underbrace{\quad}_{=3}$

$$\begin{aligned} u &= 6 \rightarrow 6.001 \\ v &= 11 \rightarrow 11.001 \\ J &= 33 \rightarrow 33.003 \end{aligned}$$

$$b = 3 \rightarrow 3.001$$

$$\begin{aligned} u &= b \cdot c = 6 \rightarrow 6.002 \\ J &= 33.006 \end{aligned}$$

$$\begin{aligned} c &= 2 \\ &1.006 \end{aligned}$$

$$\begin{aligned} v &= 11.002 \\ J &= 3v \end{aligned}$$



deeplearning.ai

# Basics of Neural Network Programming

---

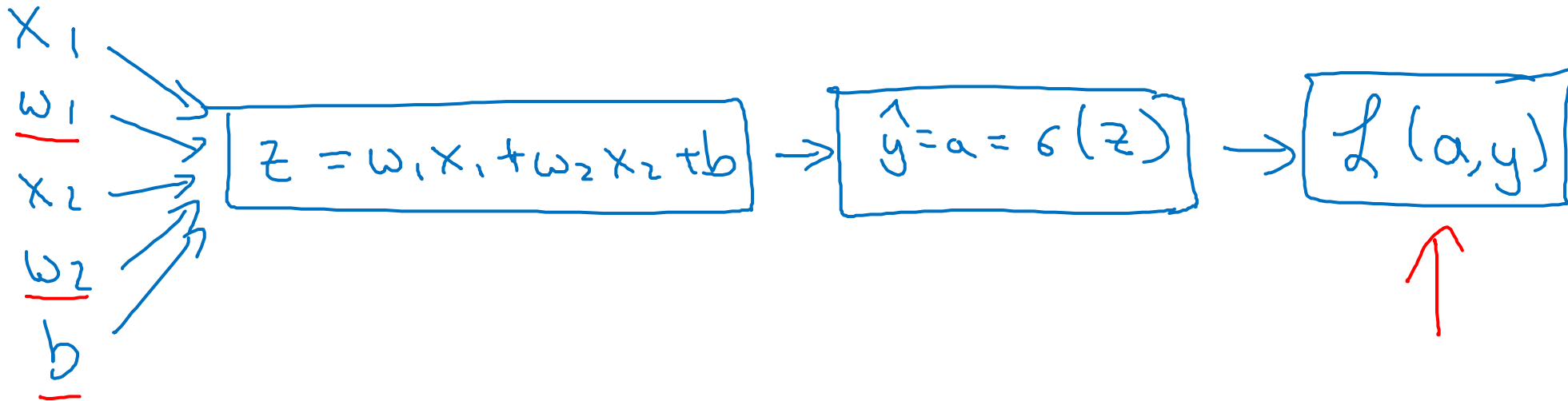
Logistic Regression  
Gradient descent

# Logistic regression recap

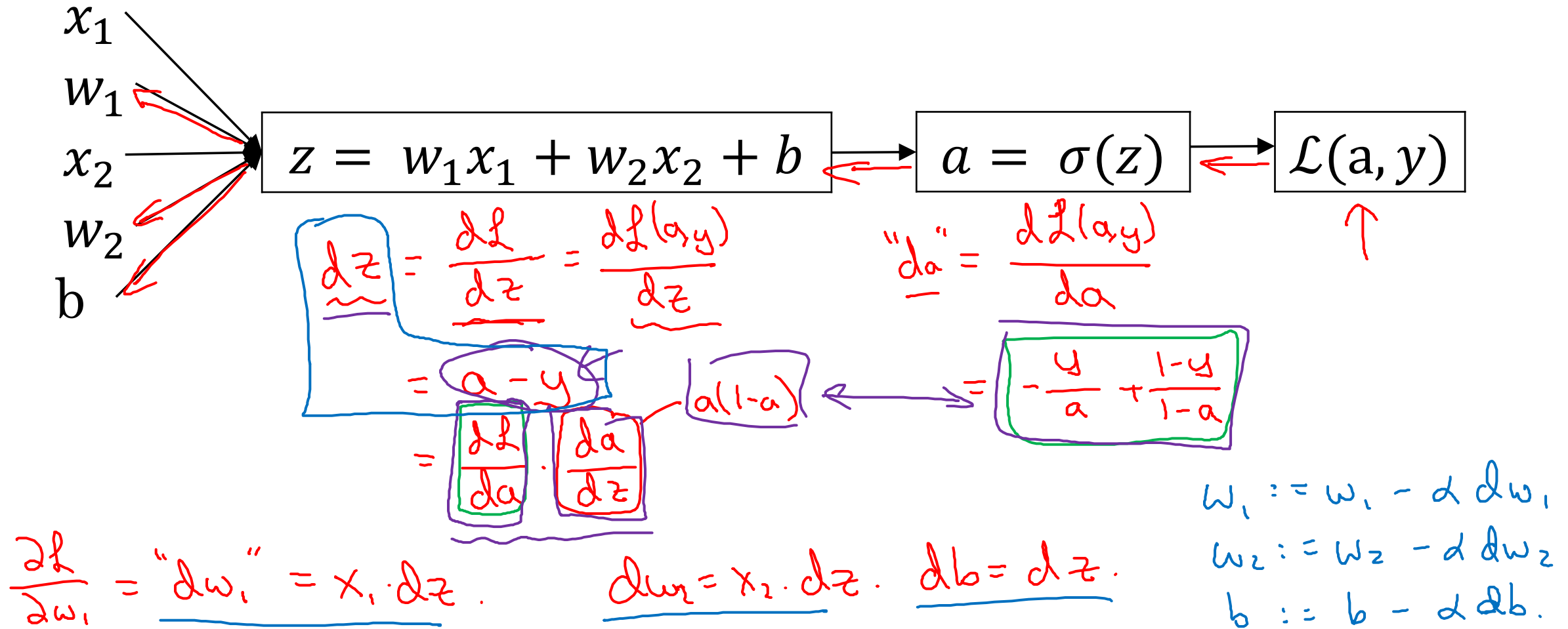
→  $z = w^T x + b$

→  $\hat{y} = a = \sigma(\underline{z})$

→  $\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$



# Logistic regression derivatives





deeplearning.ai

# Basics of Neural Network Programming

---

Gradient descent  
on  $m$  examples

# Logistic regression on $m$ examples

$$\underline{J(w, b)} = \frac{1}{m} \sum_{i=1}^m \ell(a^{(i)}, y^{(i)})$$

$$\rightarrow a^{(i)} = \hat{y}^{(i)} = \sigma(z^{(i)}) = \sigma(w^T x^{(i)} + b)$$

$$(x^{(i)}, y^{(i)})$$

$$\underline{dw_1^{(i)}}, \underline{dw_2^{(i)}}, \underline{db^{(i)}}$$

$$\underline{\frac{\partial}{\partial w_1} J(w, b)} = \frac{1}{m} \sum_{i=1}^m \underbrace{\frac{\partial}{\partial w_1} \ell(a^{(i)}, y^{(i)})}_{\underline{dw_1^{(i)}} - (x^{(i)}, y^{(i)})}$$

# Logistic regression on $m$ examples

$$J=0; \underline{dw_1}=0; \underline{dw_2}=0; \underline{db}=0$$

→ For  $i=1$  to  $m$

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += -[y^{(i)} \log a^{(i)} + (1-y^{(i)}) \log(1-a^{(i)})]$$

$$\underline{dz^{(i)}} = a^{(i)} - y^{(i)}$$

$$dw_1 += x_1^{(i)} dz^{(i)}$$

$$dw_2 += x_2^{(i)} dz^{(i)}$$

$$db += dz^{(i)}$$

$n=2$

$dw_3$   
 $\vdots$   
 $dw_n$

$J /= m \leftarrow$

$$\underset{\uparrow}{dw_1} /= m; \quad \underset{\uparrow}{dw_2} /= m; \quad \underset{\uparrow}{db} /= m. \quad \leftarrow$$

$$dw_1 = \frac{\partial J}{\partial w_1}$$

$$w_1 := w_1 - \alpha \underline{dw_1}$$

$$w_2 := w_2 - \alpha \underline{dw_2}$$

$$b := b - \alpha \underline{db}$$

Vectorization



deeplearning.ai

# Basics of Neural Network Programming

---

## Vectorization



# What is vectorization?

$$z = \underbrace{w^T x}_{\text{dot product}} + b$$

Non-vectorized:

$$z = 0$$

for  $i$  in  $\text{range}(n-x)$ :

$$z += w[i] * x[i]$$

$$z += b$$

$$w = \begin{bmatrix} : \\ : \\ : \end{bmatrix} \quad x = \begin{bmatrix} : \\ : \\ : \end{bmatrix}$$

$$w \in \mathbb{R}^{n_x}$$

$$x \in \mathbb{R}^{n_x}$$

Vectorized

$$z = \underbrace{\text{np.dot}(w, x)}_{w^T x} + b$$

$\Rightarrow$  GPU } SIMD - single instruction  
 $\Rightarrow$  CPU } multiple data.



deeplearning.ai

# Basics of Neural Network Programming

---

## More vectorization examples

# Neural network programming guideline

Whenever possible, avoid explicit for-loops.

$$u = Av$$

$$u_i = \sum_j A_{ij} v_j$$

$$u = \text{np.zeros}(n, 1)$$

for i ... ←

for j ... ←

$$u[i] += A[i][j] * v[j]$$

$$u = \text{np.dot}(A, v)$$

# Vectors and matrix valued functions

Say you need to apply the exponential operation on every element of a matrix/vector.

$$v = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} \rightarrow u = \begin{bmatrix} e^{v_1} \\ e^{v_2} \\ \vdots \\ e^{v_n} \end{bmatrix}$$

```
→ u = np.zeros((n,1))  
→ for i in range(n):  
    → u[i]=math.exp(v[i])
```

```
import numpy as np  
u = np.exp(v)  
  
np.log(v)  
np.abs(v)  
np.maximum(v, 0)  
v**2  
1/v
```

# Logistic regression derivatives

$$J = 0, \quad \boxed{\cancel{dw_1 = 0, dw_2 = 0}}, \quad db = 0$$

$$dw = np.zeros((n-x, 1))$$

→ for i = 1 to n:

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += -[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

$$dz^{(i)} = a^{(i)}(1 - a^{(i)})$$

$$\cancel{dw_1 += x_1^{(i)} dz^{(i)}}$$

$$\cancel{dw_2 += x_2^{(i)} dz^{(i)}}$$

$$db += dz^{(i)}$$

$$n_x = 2$$

$$dw += x^{(i)} dz^{(i)}$$

$$J = J/m, \quad \boxed{\cancel{dw_1 = dw_1/m, dw_2 = dw_2/m}}, \quad db = db/m$$

$$dw /= m.$$



deeplearning.ai

# Basics of Neural Network Programming

---

## Vectorizing Logistic Regression

# Vectorizing Logistic Regression

$z^{(1)} = w^T x^{(1)} + b$

$a^{(1)} = \sigma(z^{(1)})$

$$\underline{z^{(2)}} = w^T x^{(2)} + b$$
$$\underline{a^{(2)}} = \sigma(z^{(2)})$$

$$\begin{aligned} \underline{z^{(3)}} &= w^T x^{(3)} + b \\ \underline{a^{(3)}} &= \sigma(\underline{z^{(3)}}) \end{aligned}$$

$$\underline{\underline{X}} = \begin{bmatrix} | & | & & | \\ X^{(1)} & X^{(2)} & \dots & X^{(m)} \\ | & | & & | \end{bmatrix}$$

$$\frac{(n_x, m)}{\mathbb{R}^{n_x \times m}}$$

$$\begin{bmatrix} 1 & \dots & 0 \end{bmatrix} \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \dots \\ x^{(m)} \end{bmatrix}$$

$$\underline{Z} = \begin{bmatrix} \underline{z}^{(1)} & \underline{z}^{(2)} & \dots & \underline{z}^{(m)} \end{bmatrix} = \underbrace{w^T X}_{1 \times m} + \underbrace{[b \ b \dots b]}_{1 \times m} = \boxed{w^T x^{(1)} + b} \quad \boxed{w^T x^{(2)} + b} \quad \dots \quad \boxed{w^T x^{(m)} + b}$$

$$\rightarrow \underline{z = np.dot(w.T, x) + \frac{b}{N}} \quad (1.1) \quad \mathbb{R}$$

$$\underline{A} = [\underline{a^{(1)}} \quad \underline{a^{(2)}} \quad \dots \quad \underline{a^{(m)}}] = \underline{\sigma(z)}$$

"Broadcasting"



deeplearning.ai

# Basics of Neural Network Programming

---

## Vectorizing Logistic Regression's Gradient Computation



# Vectorizing Logistic Regression

$$dz^{(1)} = a^{(1)} - y^{(1)}$$

$$dz^{(2)} = a^{(2)} - y^{(2)}$$

.....

$$dz = [dz^{(1)} \quad dz^{(2)} \quad \dots \quad dz^{(m)}]$$

$1 \times m$

$$A = [a^{(1)} \quad \dots \quad a^{(m)}] \quad Y = [y^{(1)} \quad \dots \quad y^{(m)}]$$

$$\rightarrow dz = A - Y = [a^{(1)} - y^{(1)} \quad a^{(2)} - y^{(2)} \quad \dots]$$

$$\rightarrow dw = 0$$

$$dw += \frac{x^{(1)} dz^{(1)}}{m}$$

$$dw += \frac{x^{(2)} dz^{(2)}}{m}$$

$\vdots$

$$dw/ = m$$

$$db = 0$$

$$db += dz^{(1)}$$

$$db += dz^{(2)}$$

$$\vdots$$

$$db += dz^{(m)}$$

$$db/ = m$$

$$db = \frac{1}{m} \sum_{i=1}^m dz^{(i)}$$

$$= \frac{1}{m} \text{np.sum}(dz)$$

$$dw = \frac{1}{m} X dz^T$$

$$= \frac{1}{m} \begin{bmatrix} x^{(1)} & \dots & x^{(m)} \\ 1 & & 1 \end{bmatrix} \begin{bmatrix} dz^{(1)} \\ \vdots \\ dz^{(m)} \end{bmatrix}$$

$$= \frac{1}{m} \left[ \underline{x^{(1)} dz^{(1)}} + \dots + \underline{x^{(m)} dz^{(m)}} \right]$$

$n \times 1$

# Implementing Logistic Regression

$J = 0, dw_1 = 0, dw_2 = 0, db = 0$

for  $i = 1$  to  $m$ :

$$z^{(i)} = w^T x^{(i)} + b \leftarrow$$

$$a^{(i)} = \sigma(z^{(i)}) \leftarrow$$

$$J += -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)} \leftarrow$$

$$\left[ \begin{array}{l} dw_1 += x_1^{(i)} dz^{(i)} \\ dw_2 += x_2^{(i)} dz^{(i)} \end{array} \right\} dw += x^{(i)} * dz^{(i)}$$

$$db += dz^{(i)}$$

$$J = J/m, dw_1 = dw_1/m, dw_2 = dw_2/m$$

$$db = db/m$$

for iter in range(1000):  $\leftarrow$

$$Z = w^T X + b$$

$$= \text{np.dot}(w.T, X) + b$$

$$A = \sigma(Z)$$

$$dZ = A - Y$$

$$dw = \frac{1}{m} X dZ^T$$

$$db = \frac{1}{m} \text{np.sum}(dZ)$$

$$w := w - \alpha dw$$

$$b := b - \alpha db$$



deeplearning.ai

# Basics of Neural Network Programming

---

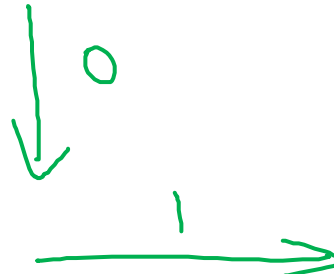
## Broadcasting in Python

# Broadcasting example

Calories from Carbs, Proteins, Fats in 100g of different foods:

|         | ↓<br>Apples | ↓<br>Beef | ↓<br>Eggs | ↓<br>Potatoes |              |
|---------|-------------|-----------|-----------|---------------|--------------|
| Carb    | 56.0        | 0.0       | 4.4       | 68.0          | = A<br>(3,4) |
| Protein | 1.2         | 104.0     | 52.0      | 8.0           |              |
| Fat     | 1.8         | 135.0     | 99.0      | 0.9           |              |

59 cal  $\frac{56}{59} \approx 94.9\%$



Calculate % of calories from Carb, Protein, Fat. Can you do this without explicit for-loop?

```
cal = A.sum(axis = 0)  
percentage = 100 * A / (cal.reshape(1,4))
```

↑ (3,4) / (1,4)

# Broadcasting example

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} + \begin{bmatrix} 100 \\ 100 \\ 100 \\ 100 \end{bmatrix} \quad \text{100}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 100 & 200 & 300 \\ 100 & 200 & 300 \end{bmatrix}$$

$(m,n) \quad (2,3) \quad (1,n) \rightsquigarrow (m,n) \quad (2,3)$

↓   ↓   ↓

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 100 & 100 & 100 \\ 200 & 200 & 200 \end{bmatrix} =$$

$(m,n) \quad (m,1) \rightsquigarrow (m,n)$

←   ←

# General Principle

$$\begin{array}{ccc} (m, n) & + & (1, n) \\ \text{matrix} & \times & \rightsquigarrow (m, n) \\ \hline & / & \end{array}$$

$$(m, 1) \rightsquigarrow (m, n)$$

$$(m, 1)$$

+

$$\mathbb{R}$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

+

$$100$$

$$= \begin{bmatrix} 101 \\ 102 \\ 103 \end{bmatrix}$$

$$[1 \ 2 \ 3]$$

+

$$100$$

$$= [101 \quad 102 \quad 103]$$

Matlab/Octave: bsxfun



deeplearning.ai

# Basics of Neural Network Programming

---

Explanation of logistic  
regression cost function  
(Optional)

# Logistic regression cost function

$$\hat{y} = \sigma(w^T x + b) \quad \text{where} \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpret  $\hat{y} = p(y=1|x)$

If  $y=1$  :  $p(y|x) = \hat{y}$

If  $y=0$  :  $\underline{p(y|x)} = \underline{1 - \hat{y}}$



# Logistic regression cost function

$$\begin{aligned} \rightarrow & \text{If } y = 1: p(y|x) = \hat{y} \\ \rightarrow & \text{If } y = 0: p(y|x) = 1 - \hat{y} \end{aligned} \quad \left. \vphantom{\begin{aligned} \rightarrow & \text{If } y = 1: p(y|x) = \hat{y} \\ \rightarrow & \text{If } y = 0: p(y|x) = 1 - \hat{y} \end{aligned}} \right\} p(y|x)$$

$$p(y|x) = \hat{y}^y (1-\hat{y})^{(1-y)} \quad \leftarrow$$

$$\text{If } y=1: p(y|x) = \hat{y} \underbrace{(1-\hat{y})^0}_{=1}$$

$$\text{If } y=0: p(y|x) = \hat{y}^0 \underbrace{(1-\hat{y})^{(1-0)}}_{=1} = 1 \times (1-\hat{y}) = \underline{1-\hat{y}}$$

$$\begin{aligned} \uparrow \log p(y|x) &= \log \hat{y}^y (1-\hat{y})^{(1-y)} = y \log \hat{y} + (1-y) \log (1-\hat{y}) \\ &= - \underbrace{\ell(\hat{y}, y)}_{\downarrow} \end{aligned}$$

# Cost on $m$ examples

$$\log p(\text{labels in training set}) = \log \prod_{i=1}^m p(y^{(i)} | x^{(i)}) \quad \leftarrow$$

$$\log p(\text{-----}) = \sum_{i=1}^m \underbrace{\log p(y^{(i)} | x^{(i)})}_{- \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}$$

$$= - \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Maximum likelihood  
estimator  $\nearrow$

Cost:  $\underbrace{J(w, b)}_{\text{(minimize)}} = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$

# Standard notations for Deep Learning

This document has the purpose of discussing a new standard for deep learning mathematical notations.

## 1 Neural Networks Notations.

### General comments:

- superscript (i) will denote the  $i^{th}$  training example while superscript [l] will denote the  $l^{th}$  layer

### Sizes:

- $m$  : number of examples in the dataset
- $n_x$  : input size
- $n_y$  : output size (or number of classes)
- $n_h^{[l]}$  : number of hidden units of the  $l^{th}$  layer

In a for loop, it is possible to denote  $n_x = n_h^{[0]}$  and  $n_y = n_h^{[\text{number of layers} + 1]}$ .

- $L$  : number of layers in the network.

### Objects:

- $X \in \mathbb{R}^{n_x \times m}$  is the input matrix
- $x^{(i)} \in \mathbb{R}^{n_x}$  is the  $i^{th}$  example represented as a column vector

- $Y \in \mathbb{R}^{n_y \times m}$  is the label matrix

- $y^{(i)} \in \mathbb{R}^{n_y}$  is the output label for the  $i^{th}$  example

- $W^{[l]} \in \mathbb{R}^{\text{number of units in next layer} \times \text{number of units in the previous layer}}$  is the weight matrix, superscript [l] indicates the layer

- $b^{[l]} \in \mathbb{R}^{\text{number of units in next layer}}$  is the bias vector in the  $l^{th}$  layer

- $\hat{y} \in \mathbb{R}^{n_y}$  is the predicted output vector. It can also be denoted  $a^{[L]}$  where  $L$  is the number of layers in the network.

### Common forward propagation equation examples:

$a = g^{[l]}(W_x x^{(i)} + b_1) = g^{[l]}(z_1)$  where  $g^{[l]}$  denotes the  $l^{th}$  layer activation function

$$\hat{y}^{(i)} = \text{softmax}(W_h h + b_2)$$

- General Activation Formula:  $a_j^{[l]} = g^{[l]}(\sum_k w_{jk}^{[l]} a_k^{[l-1]} + b_j^{[l]}) = g^{[l]}(z_j^{[l]})$

- $J(x, W, b, y)$  or  $J(\hat{y}, y)$  denote the cost function.

### Examples of cost function:

$$J_{CE}(\hat{y}, y) = - \sum_{i=0}^m y^{(i)} \log \hat{y}^{(i)}$$

$$J_1(\hat{y}, y) = \sum_{i=0}^m |y^{(i)} - \hat{y}^{(i)}|$$

## 2 Deep Learning representations

**For representations:**

- nodes represent inputs, activations or outputs
- edges represent weights or biases

Here are several examples of Standard deep learning representations

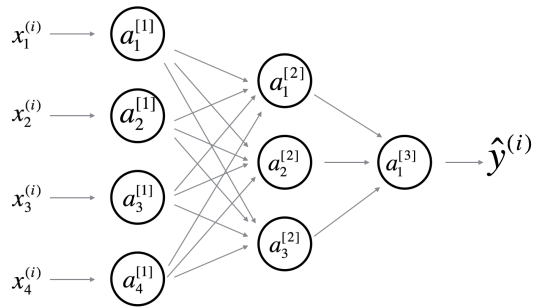


Figure 1: Comprehensive Network: representation commonly used for Neural Networks. For better aesthetic, we omitted the details on the parameters ( $w_{ij}^{[l]}$  and  $b_i^{[l]}$  etc...) that should appear on the edges

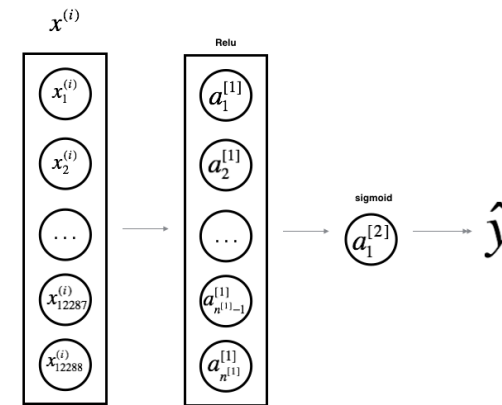


Figure 2: Simplified Network: a simpler representation of a two layer neural network, both are equivalent.

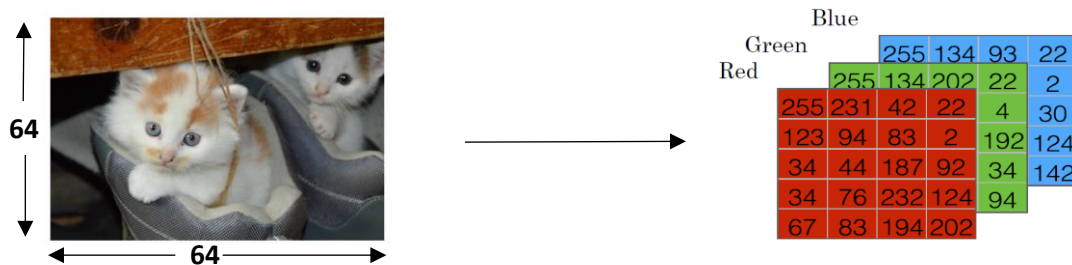
# Binary Classification

In a binary classification problem, the result is a discrete value output.

For example     - account hacked (1) or not hacked (0)  
                   - a tumor malign (1) or benign (0)

Example: Cat vs Non-Cat

The goal is to train a classifier for which the input is an image represented by a feature vector,  $x$ , and predicts whether the corresponding label is 1 or 0. In this case, whether this is a cat image (1) or a non-cat image (0).



An image is stored in the computer in three separate matrices corresponding to the Red, Green, and Blue color channels of the image. The three matrices have the same size as the image, for example, the resolution of the cat image is 64 pixels X 64 pixels, the three matrices (RGB) are 64 X 64 each.

The value in a cell represents the pixel intensity which will be used to create a feature vector of  $n$ -dimension. In pattern recognition and machine learning, a feature vector represents an image, Then the classifier's job is to determine whether it contain a picture of a cat or not.

To create a feature vector,  $x$ , the pixel intensity values will be “unrolled” or “reshaped” for each color. The dimension of the input feature vector  $x$  is  $n = 64 \times 64 \times 3 = 12288$ .

$$x = \begin{bmatrix} 255 \\ 231 \\ 42 \\ \vdots \\ 255 \\ 134 \\ 202 \\ \vdots \\ 255 \\ 134 \\ 93 \\ \vdots \end{bmatrix} \begin{matrix} \text{red} \\ \text{green} \\ \text{blue} \end{matrix}$$

## Logistic Regression

Logistic regression is a learning algorithm used in a supervised learning problem when the output  $y$  are all either zero or one. The goal of logistic regression is to minimize the error between its predictions and training data.

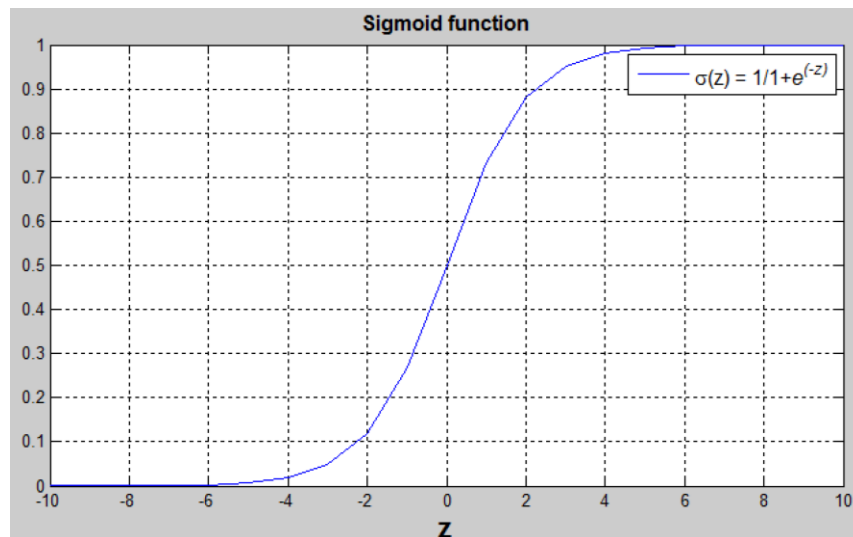
Example: Cat vs No - cat

Given an image represented by a feature vector  $x$ , the algorithm will evaluate the probability of a cat being in that image.

$$\text{Given } x, \hat{y} = P(y = 1|x), \text{ where } 0 \leq \hat{y} \leq 1$$

The parameters used in Logistic regression are:

- The input features vector:  $x \in \mathbb{R}^{n_x}$ , where  $n_x$  is the number of features
- The training label:  $y \in \{0,1\}$
- The weights:  $w \in \mathbb{R}^{n_x}$ , where  $n_x$  is the number of features
- The threshold:  $b \in \mathbb{R}$
- The output:  $\hat{y} = \sigma(w^T x + b)$
- Sigmoid function:  $s = \sigma(w^T x + b) = \sigma(z) = \frac{1}{1 + e^{-z}}$



$(w^T x + b)$  is a linear function ( $ax + b$ ), but since we are looking for a probability constraint between  $[0,1]$ , the sigmoid function is used. The function is bounded between  $[0,1]$  as shown in the graph above.

Some observations from the graph:

- If  $z$  is a large positive number, then  $\sigma(z) = 1$
- If  $z$  is small or large negative number, then  $\sigma(z) = 0$
- If  $z = 0$ , then  $\sigma(z) = 0.5$

## Logistic Regression: Cost Function

To train the parameters  $w$  and  $b$ , we need to define a cost function.

Recap:

$$\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b), \text{ where } \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

$x^{(i)}$  the i-th training example

Given  $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$ , we want  $\hat{y}^{(i)} \approx y^{(i)}$

Loss (error) function:

The loss function measures the discrepancy between the prediction ( $\hat{y}^{(i)}$ ) and the desired output ( $y^{(i)}$ ). In other words, the loss function computes the error for a single training example.

$$L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$$

$$L(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

- If  $y^{(i)} = 1$ :  $L(\hat{y}^{(i)}, y^{(i)}) = -\log(\hat{y}^{(i)})$  where  $\log(\hat{y}^{(i)})$  and  $\hat{y}^{(i)}$  should be close to 1
- If  $y^{(i)} = 0$ :  $L(\hat{y}^{(i)}, y^{(i)}) = -\log(1 - \hat{y}^{(i)})$  where  $\log(1 - \hat{y}^{(i)})$  and  $\hat{y}^{(i)}$  should be close to 0

Cost function

The cost function is the average of the loss function of the entire training set. We are going to find the parameters  $w$  and  $b$  that minimize the overall cost function.

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$



deeplearning.ai

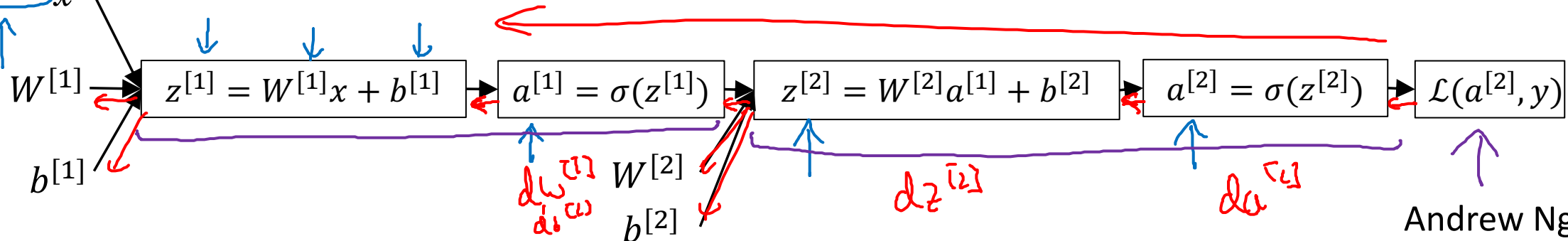
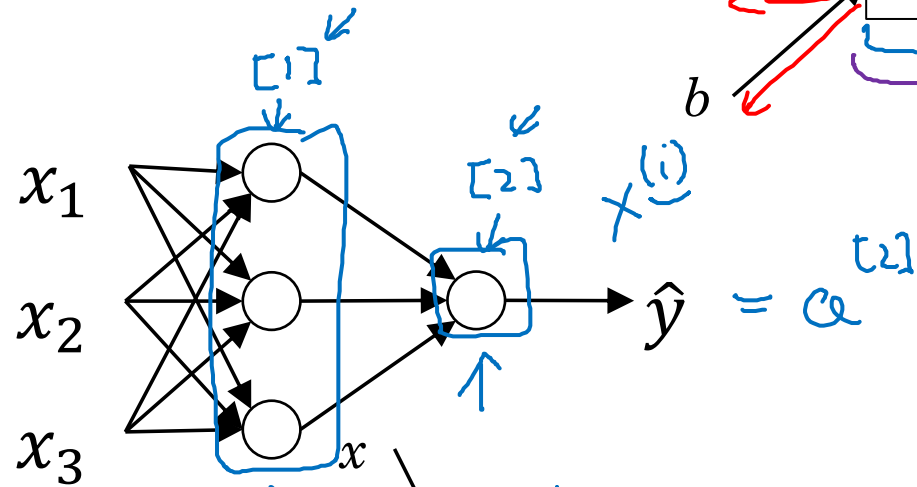
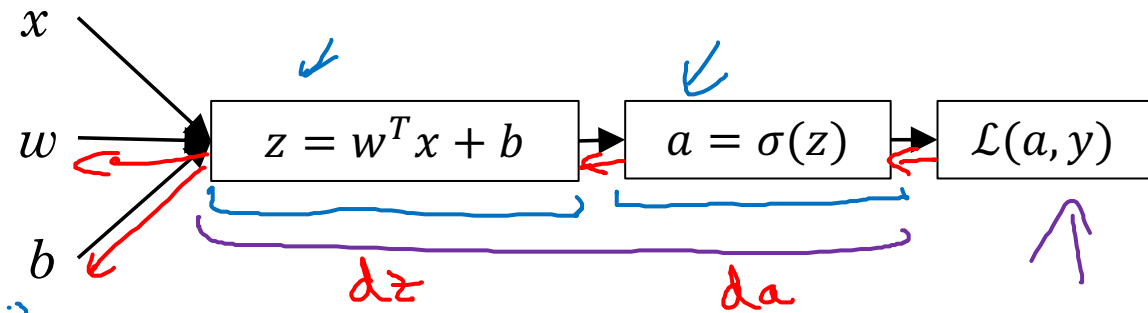
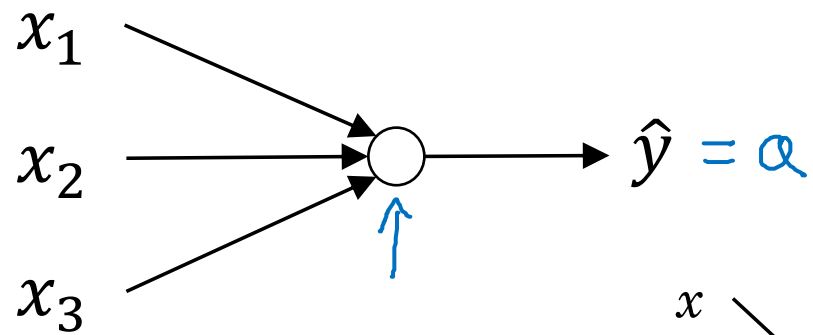
One hidden layer  
Neural Network

---

# Neural Networks Overview



# What is a Neural Network?





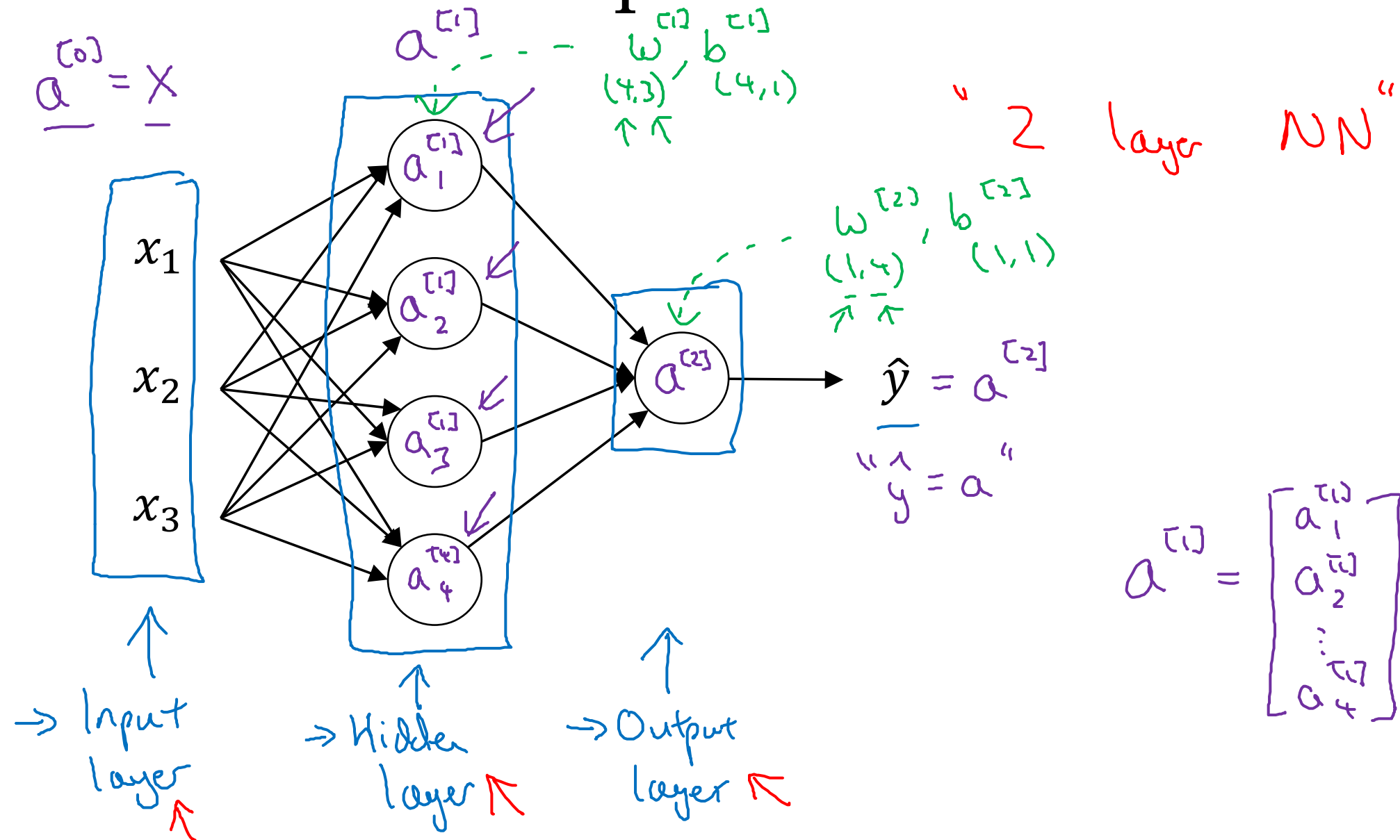
deeplearning.ai

One hidden layer  
Neural Network

---

Neural Network  
Representation

# Neural Network Representation





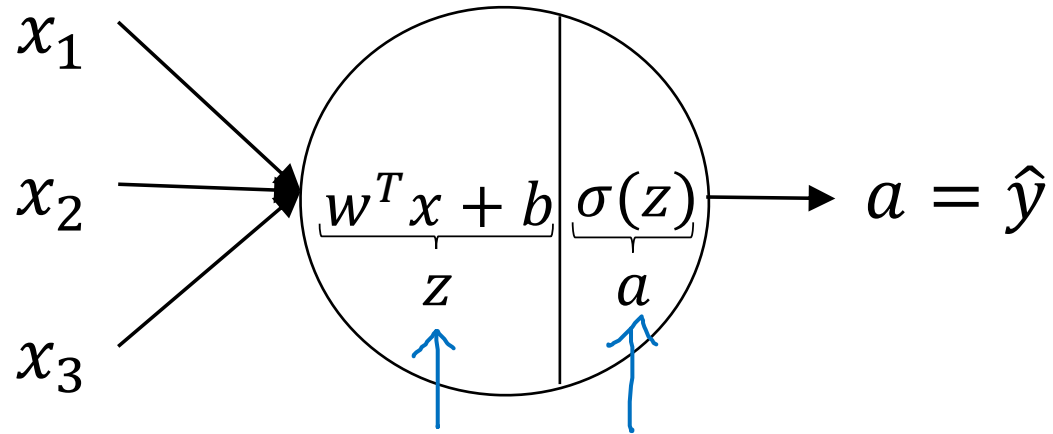
deeplearning.ai

# One hidden layer Neural Network

---

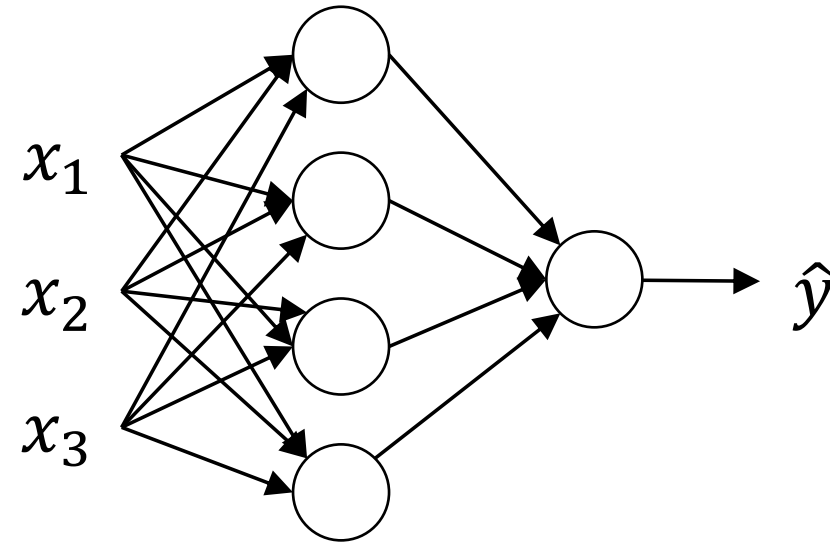
Computing a  
Neural Network's  
Output

# Neural Network Representation

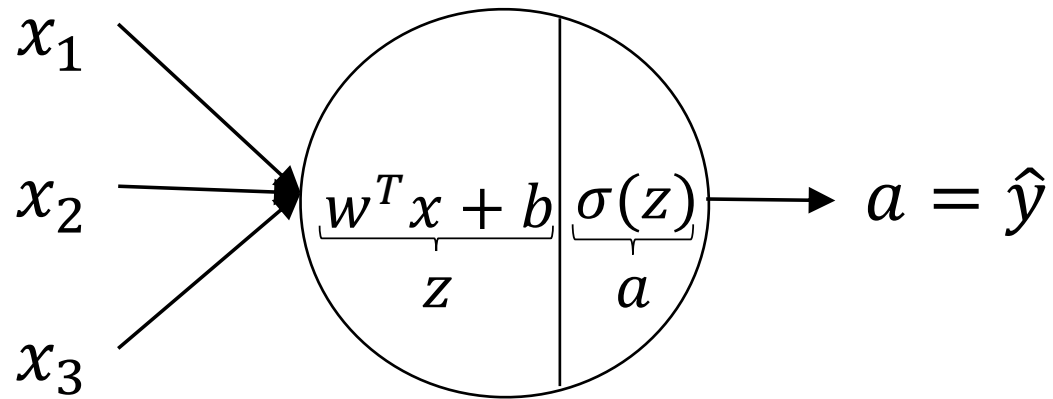


$$z = w^T x + b$$

$$a = \sigma(z)$$

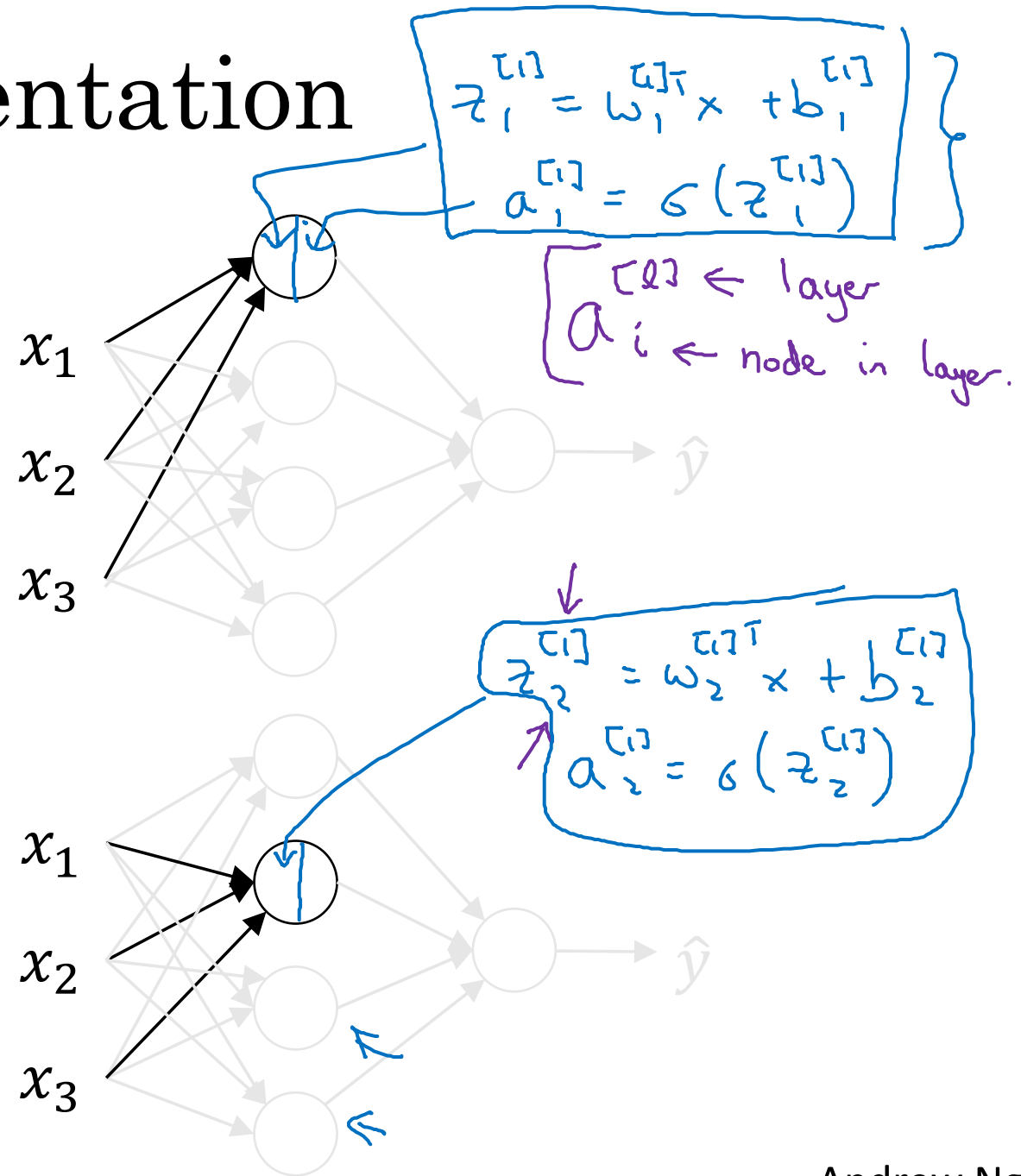


# Neural Network Representation

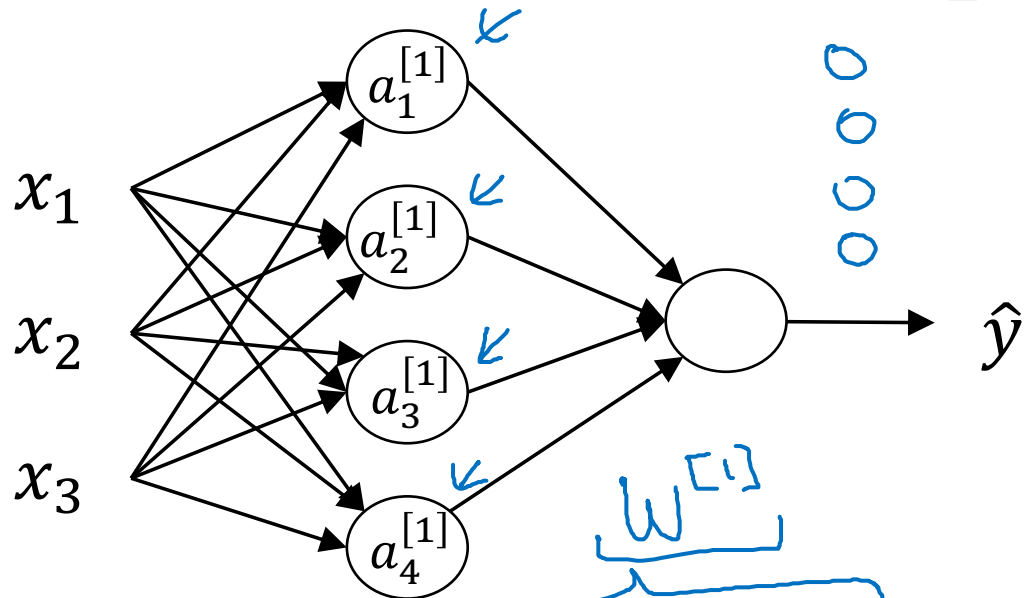


$$z = w^T x + b$$

$$a = \sigma(z)$$



# Neural Network Representation



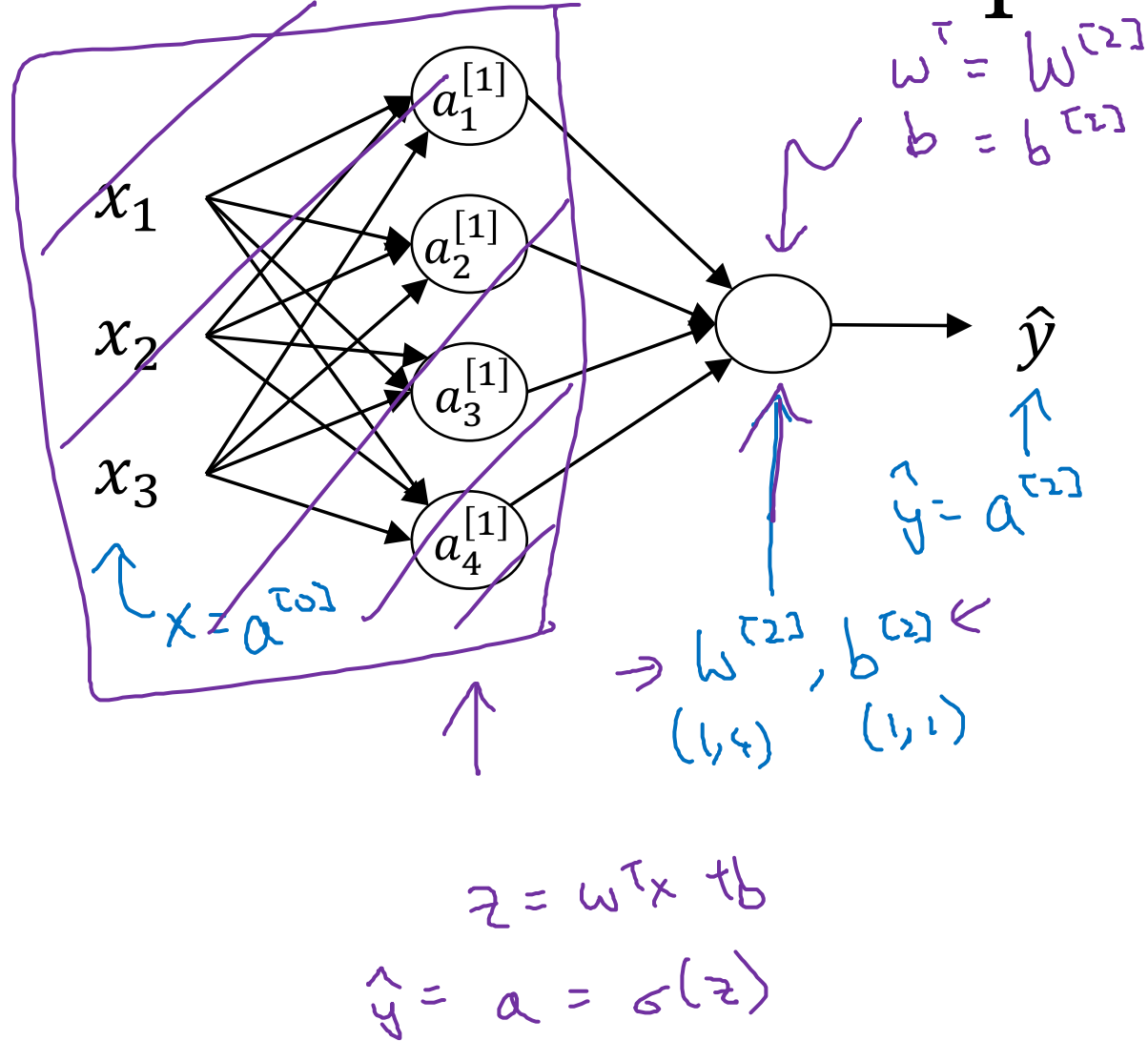
$$\begin{aligned}
 z_1^{[1]} &= w_1^{[1]T} x + b_1^{[1]} & a_1^{[1]} &= \sigma(z_1^{[1]}) \\
 z_2^{[1]} &= w_2^{[1]T} x + b_2^{[1]} & a_2^{[1]} &= \sigma(z_2^{[1]}) \\
 z_3^{[1]} &= w_3^{[1]T} x + b_3^{[1]} & a_3^{[1]} &= \sigma(z_3^{[1]}) \\
 z_4^{[1]} &= w_4^{[1]T} x + b_4^{[1]} & a_4^{[1]} &= \sigma(z_4^{[1]})
 \end{aligned}$$

Handwritten notes:  $(w_1^{[1]})^T x$  and  $Q^{[1]}$  are written above the first equation. A red box highlights the activation equations. A blue arrow points from the activation equations down to the vector  $z^{[1]}$ .

$$\begin{aligned}
 &\rightarrow z^{[1]} = \begin{bmatrix} -w_1^{[1]T} \\ -w_2^{[1]T} \\ -w_3^{[1]T} \\ -w_4^{[1]T} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix} = \begin{bmatrix} \rightarrow w_1^{[1]T} x + b_1^{[1]} \\ \rightarrow w_2^{[1]T} x + b_2^{[1]} \\ \rightarrow w_3^{[1]T} x + b_3^{[1]} \\ \rightarrow w_4^{[1]T} x + b_4^{[1]} \end{bmatrix} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} \\
 &\rightarrow a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ \vdots \\ a_4^{[1]} \end{bmatrix} = \sigma(z^{[1]})
 \end{aligned}$$

Handwritten notes:  $(4, 3)$  is written below the weight matrix.  $b^{[1]} (4, 1)$  is written below the bias vector. A red box highlights the activation vector  $a^{[1]}$  and its definition.

# Neural Network Representation learning



Given input  $x$ :

$$\begin{aligned} \rightarrow z^{[1]} &= W^{[1]} a^{[0]} + b^{[1]} \\ &\quad (4,1) \quad (4,3) \quad (3,1) \quad (4,1) \\ \rightarrow a^{[1]} &= \sigma(z^{[1]}) \\ &\quad (4,1) \quad (4,1) \\ \rightarrow z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\ &\quad (1,1) \quad (1,4) \quad (4,1) \quad (1,1) \\ \rightarrow a^{[2]} &= \sigma(z^{[2]}) \\ &\quad (1,1) \quad (1,1) \end{aligned}$$





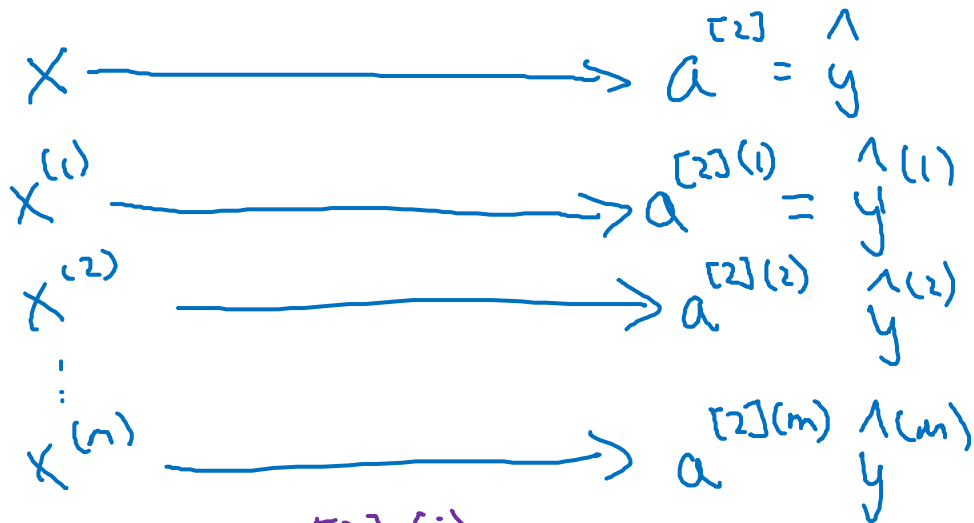
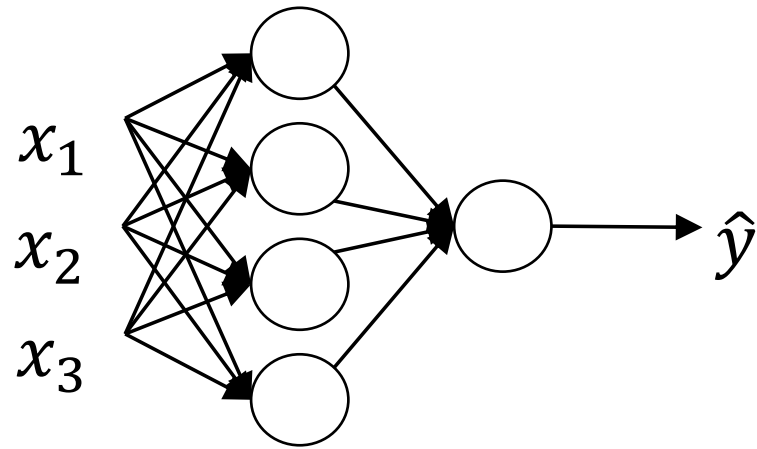
deeplearning.ai

One hidden layer  
Neural Network

---

Vectorizing across  
multiple examples

# Vectorizing across multiple examples



$a^{[2](i)}$   
 $\nwarrow \nearrow$  example  $i$   
 layer 2

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

for  $i = 1$  to  $m$ ,

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

# Vectorizing across multiple examples

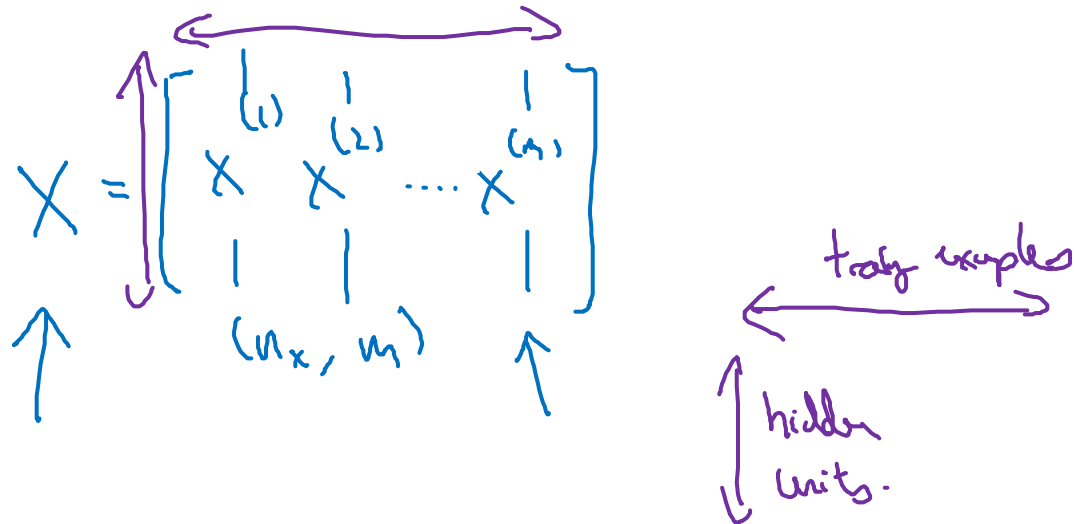
for  $i = 1$  to  $m$ :

$$z^{[1]}(i) = W^{[1]}x^{(i)} + b^{[1]}$$

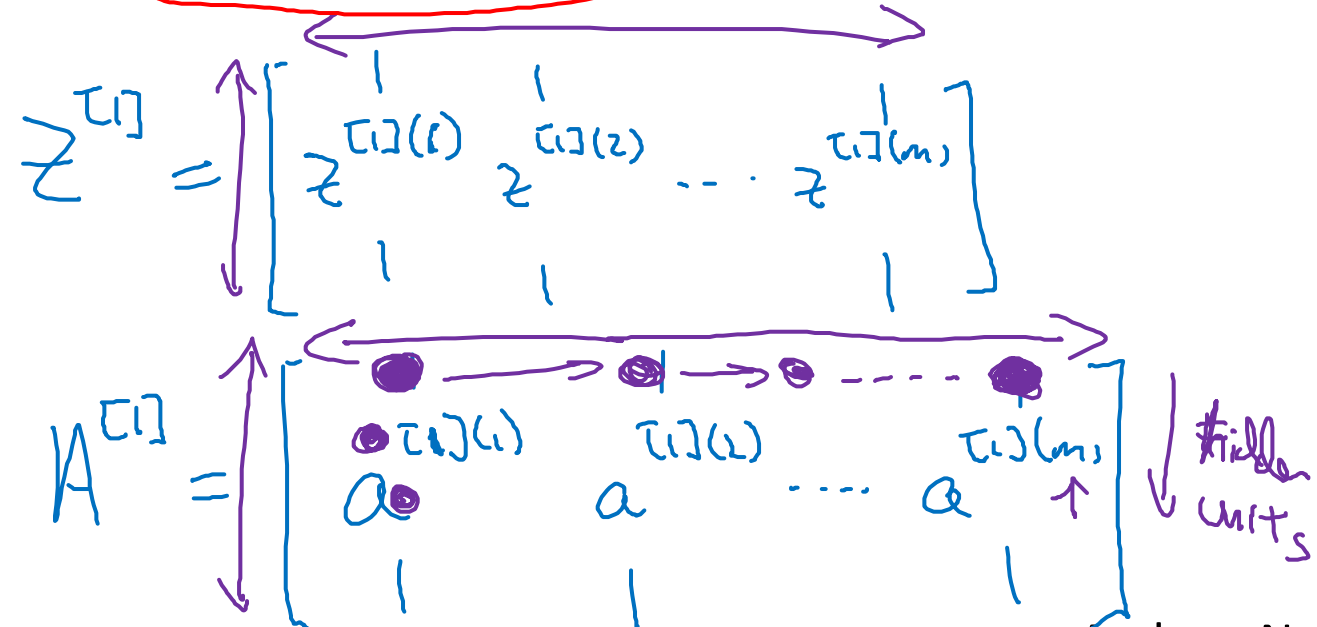
$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$z^{[2]}(i) = W^{[2]}a^{[1]}(i) + b^{[2]}$$

$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$



$$\begin{aligned} z^{[1]} &= W^{[1]}X + b^{[1]} \\ \rightarrow A^{[1]} &= \sigma(z^{[1]}) \\ \rightarrow z^{[2]} &= W^{[2]}A^{[1]} + b^{[2]} \\ \rightarrow A^{[2]} &= \sigma(z^{[2]}) \end{aligned}$$





deeplearning.ai

# One hidden layer Neural Network

---

Explanation  
for vectorized  
implementation

# Justification for vectorized implementation

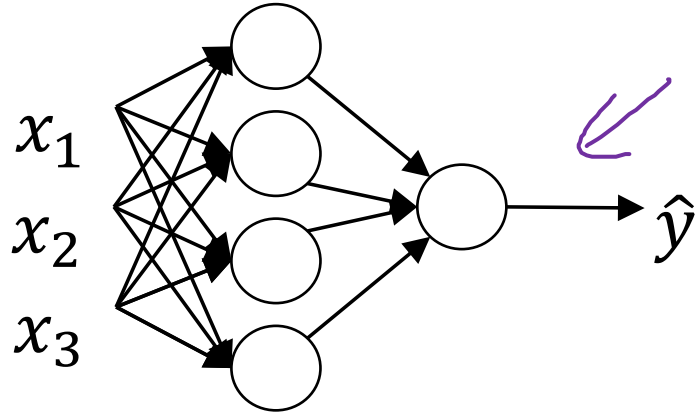
$$z^{[1](1)} = \underbrace{w^{[1]} x^{(1)}}_{\text{purple}} + \cancel{b^{[1]}}_{\text{red}}, \quad z^{[1](2)} = \underbrace{w^{[1]} x^{(2)}}_{\text{green}} + \cancel{b^{[1]}}_{\text{red}}, \quad z^{[1](3)} = \underbrace{w^{[1]} x^{(3)}}_{\text{yellow}} + \cancel{b^{[1]}}_{\text{red}}$$

$$w^{[1]} = \begin{bmatrix} \text{---} \\ \text{---} \\ \text{---} \\ \text{---} \end{bmatrix} \quad w^{[1]} x^{(1)} = \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix} \quad w^{[1]} x^{(2)} = \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix} \quad w^{[1]} x^{(3)} = \begin{bmatrix} \bullet \\ \bullet \\ \bullet \\ \bullet \end{bmatrix}$$

$$w^{[1]} \begin{bmatrix} | & | & | & \dots \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots \\ | & | & | & \dots \end{bmatrix} = \begin{bmatrix} \bullet & \bullet & \bullet & \dots \\ \bullet & \bullet & \bullet & \dots \\ \bullet & \bullet & \bullet & \dots \\ \bullet & \bullet & \bullet & \dots \end{bmatrix} = \begin{bmatrix} | & | & | & \dots \\ z^{[1](1)} & z^{[1](2)} & z^{[1](3)} & \dots \\ | & | & | & \dots \end{bmatrix} = z^{[1]}$$

$\hat{x} = X$   
 $z^{[1]} = w^{[1]} X + b^{[1]}$   
 $w^{[1]} x^{(i)} = z^{[1](i)}$

# Recap of vectorizing across multiple examples



$$X = \begin{bmatrix} | & | & \dots & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & \dots & | \end{bmatrix}$$

A purple arrow points from the matrix  $X$  towards the right.

$$\underline{A^{[1]}} = \begin{bmatrix} | & | & \dots & | \\ a^{[1]}(1) & a^{[1]}(2) & \dots & a^{[1]}(m) \\ | & | & \dots & | \end{bmatrix}$$

A purple arrow points from the matrix  $A^{[1]}$  towards the right.

for  $i = 1$  to  $m$

$$\rightarrow z^{[1]}(i) = W^{[1]}x^{(i)} + b^{[1]}$$

$$\rightarrow a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$\rightarrow z^{[2]}(i) = W^{[2]}a^{[1]}(i) + b^{[2]}$$

$$\rightarrow a^{[2]}(i) = \sigma(z^{[2]}(i))$$

$$Z^{[1]} = W^{[1]} \underline{X} + b^{[1]}$$

A blue circle is drawn around  $X$  in the equation above.

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

$$x = a^{[0]} \quad x^{(i)} = a^{[0]}(i)$$

$$W^{[1]}A^{[0]} + b^{[1]}$$



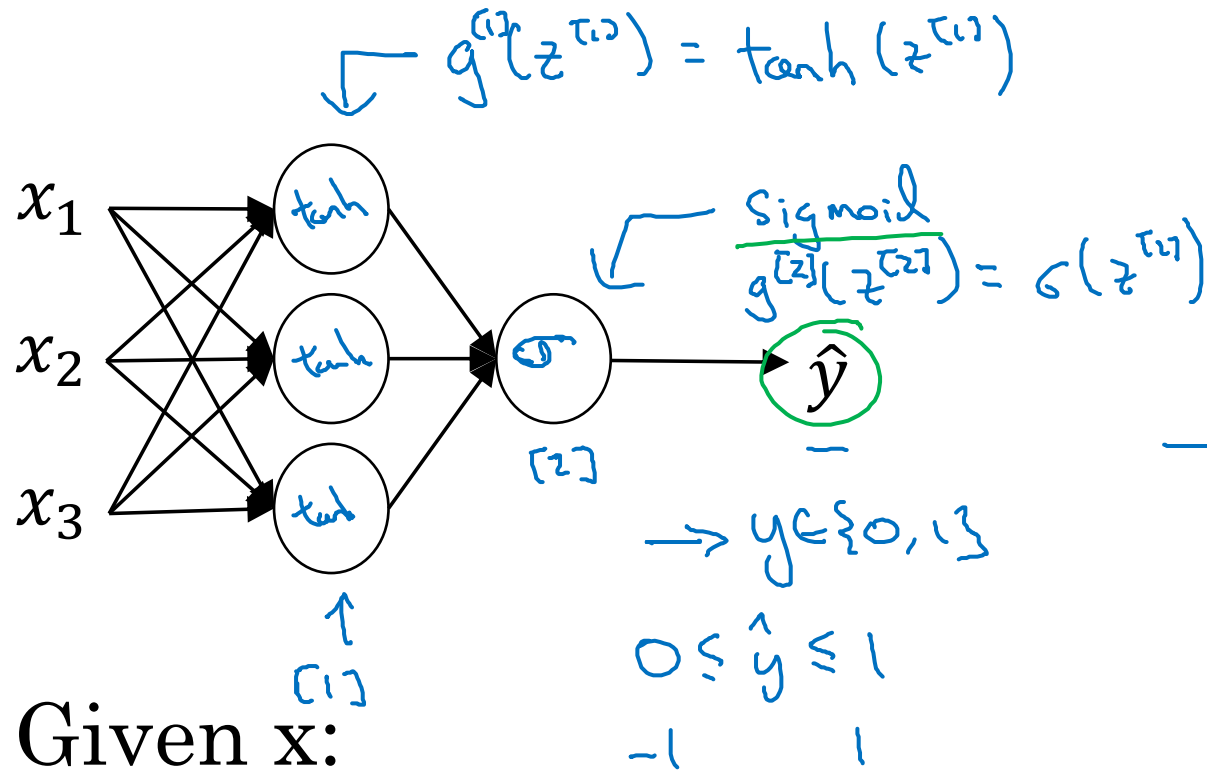
deeplearning.ai

# One hidden layer Neural Network

---

## Activation functions

# Activation functions



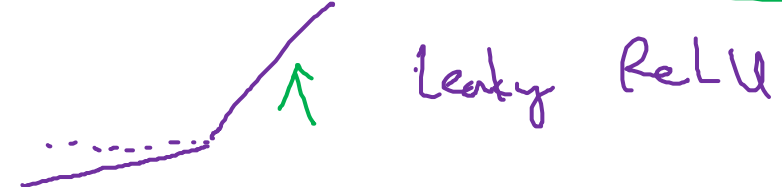
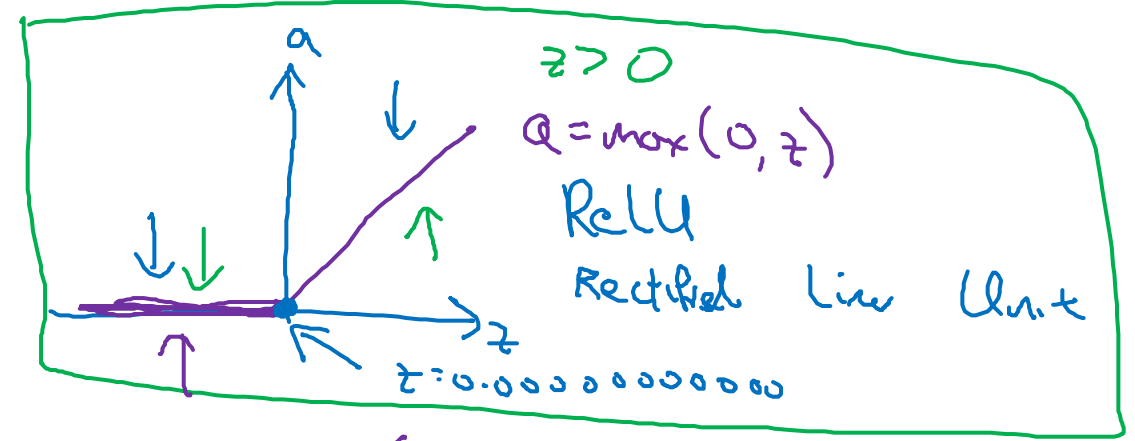
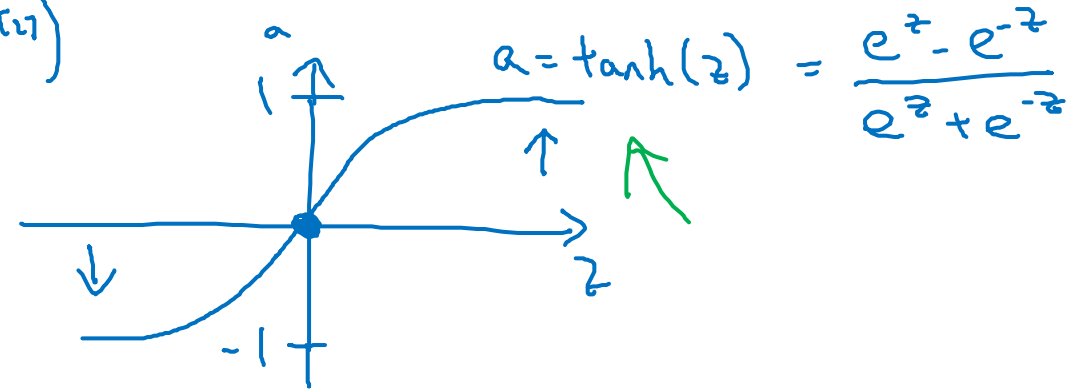
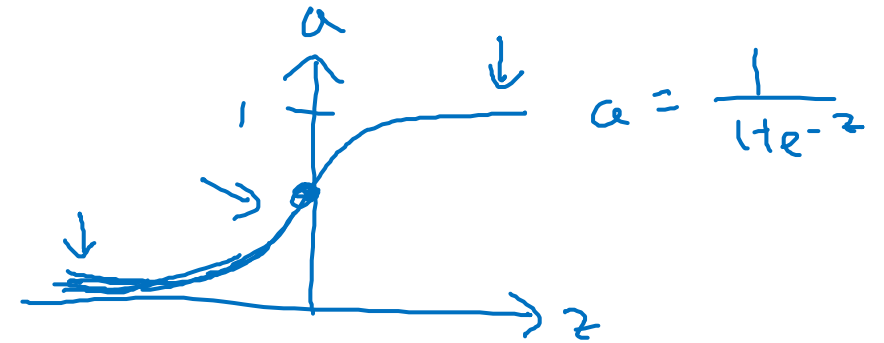
Given  $x$ :

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$\rightarrow a^{[1]} = \cancel{\sigma(z^{[1]})} g^{(1)}(z^{(1)})$$

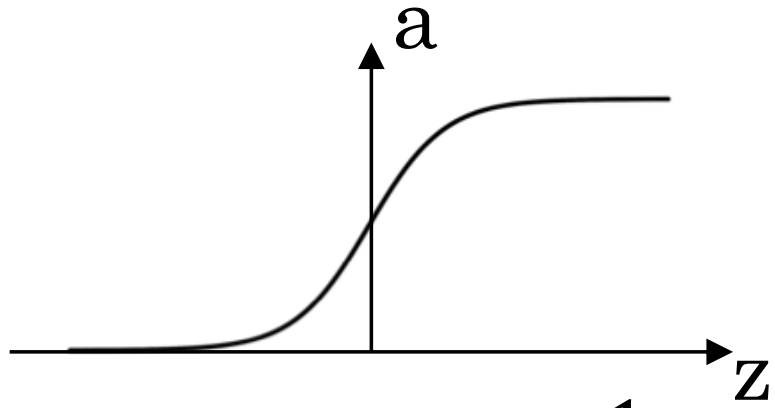
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$\rightarrow a^{[2]} = \cancel{\sigma(z^{[2]})} g^{(2)}(z^{(2)})$$

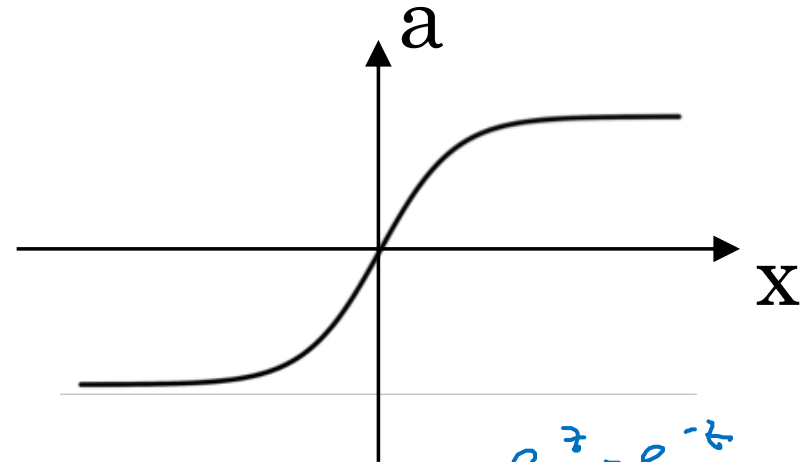




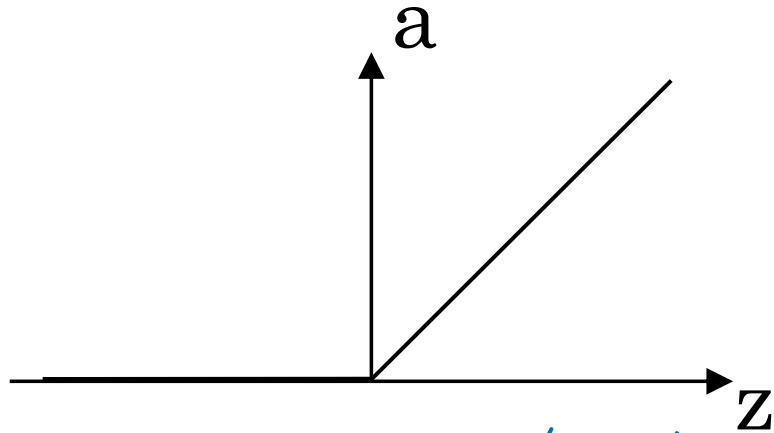
# Pros and cons of activation functions



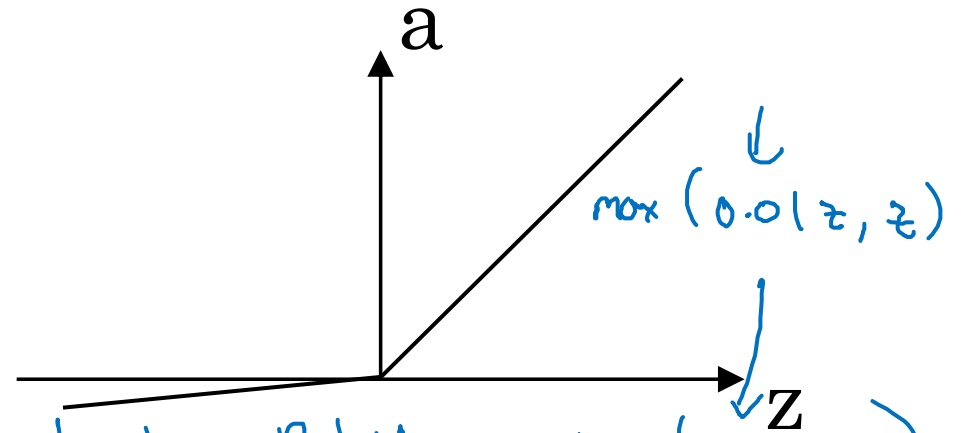
sigmoid:  $a = \frac{1}{1 + e^{-z}}$



tanh:  $a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$



ReLU  $a = \max(0, z)$



Leaky ReLU  $a = \max(0.01z, z)$



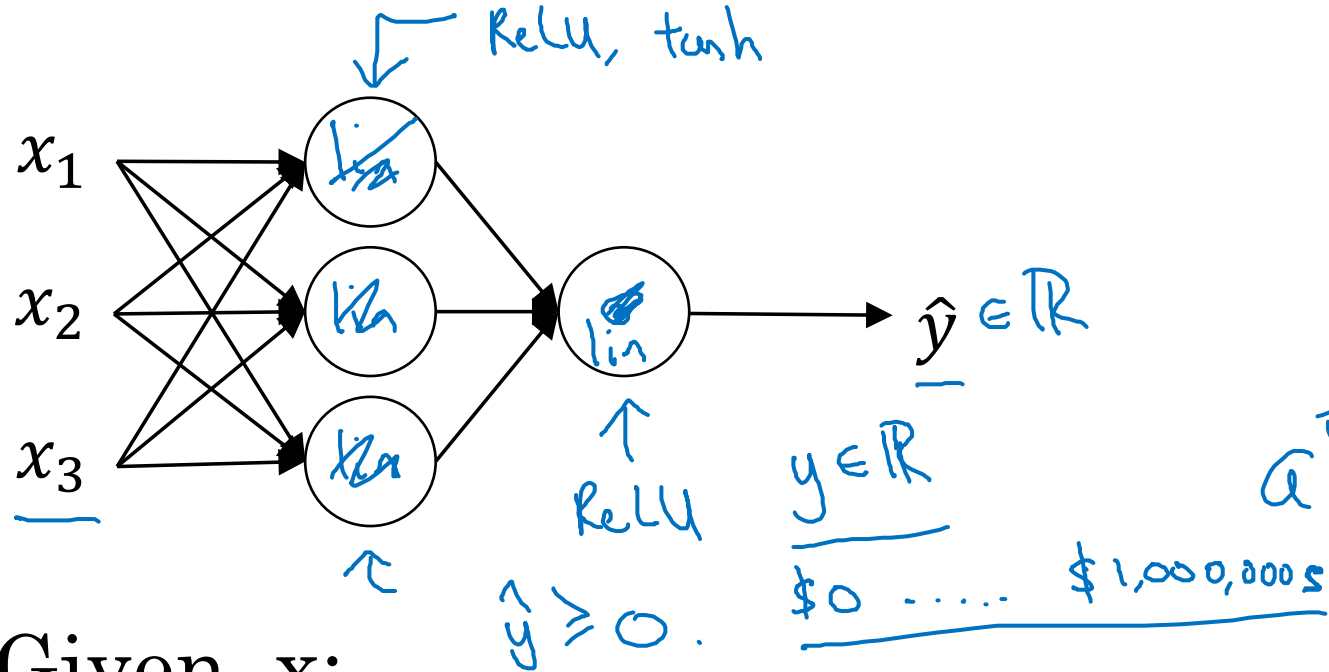
deeplearning.ai

# One hidden layer Neural Network

---

Why do you  
need non-linear  
activation functions?

# Activation function



Given  $x$ :

$$\begin{aligned} \rightarrow z^{[1]} &= W^{[1]}x + b^{[1]} \\ \rightarrow a^{[1]} &= \cancel{g^{[1]}(z^{[1]})} z^{[1]} \\ \rightarrow z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\ \rightarrow a^{[2]} &= \cancel{g^{[2]}(z^{[2]})} z^{[2]} \end{aligned}$$

$g(z) = z$   
"linear activation function"

$$\begin{aligned} a^{[1]} = z^{[1]} &= W^{[1]}x + b^{[1]} \\ a^{[2]} = z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \end{aligned}$$

$$a^{[2]} = W^{[2]} \left( W^{[1]}x + b^{[1]} \right) + b^{[2]}$$

$a^{[1]}$

$$\begin{aligned} &= \underbrace{(W^{[2]} W^{[1]})}_w x + \underbrace{(W^{[2]} b^{[1]} + b^{[2]})}_{b'} \\ &= \underline{w'x + b'} \end{aligned}$$

$$g(z) = z$$



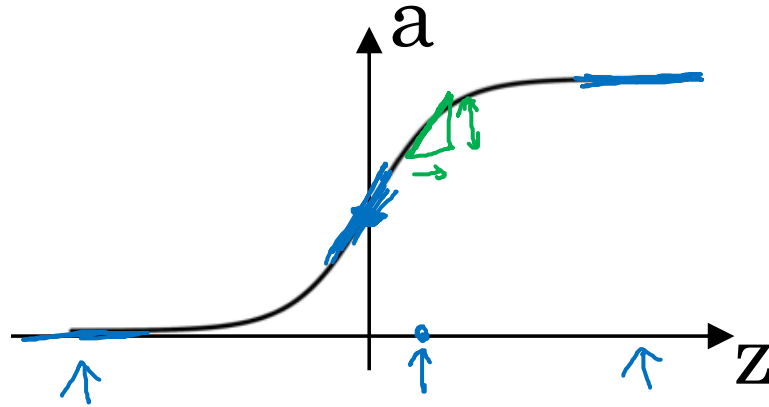
deeplearning.ai

One hidden layer  
Neural Network

---

Derivatives of  
activation functions

# Sigmoid activation function



$$\underline{g(z) = \frac{1}{1 + e^{-z}}}$$

$$a = g(z) = \frac{1}{1 + e^{-z}}$$

$$\boxed{g'(z)} = \boxed{\frac{d}{dz} g(z)} = \text{slope of } g(z) \text{ at } z$$

$$= \frac{1}{1 + e^{-z}} \left( 1 - \frac{1}{1 + e^{-z}} \right)$$

$$= g(z) (1 - g(z)) \leftarrow$$

$$= \boxed{a(1-a)} \quad \left| \begin{array}{l} g'(z) = a(1-a) \\ \uparrow \\ a \end{array} \right.$$

$$z = 10. \quad g(z) \approx 1$$

$$\frac{d}{dz} g(z) \approx 1(1-1) \approx 0$$

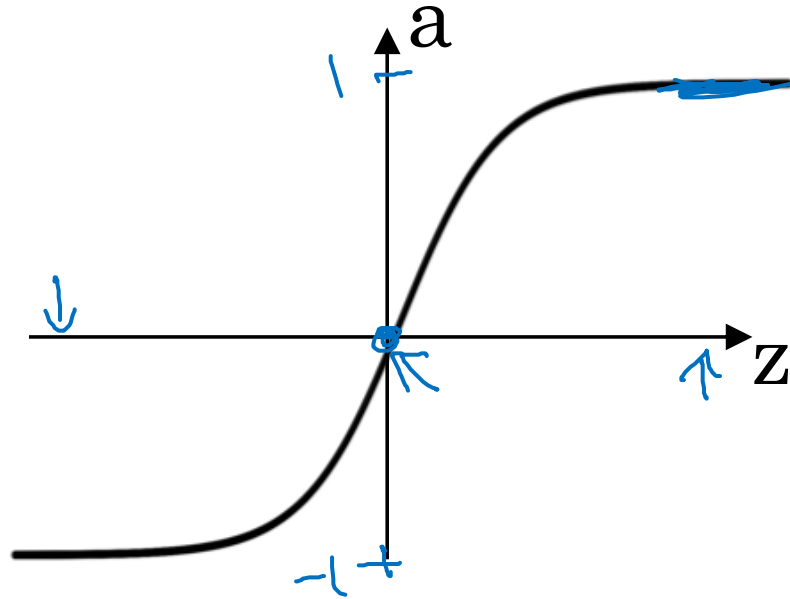
$$z = -10 \quad g(z) \approx 0$$

$$\frac{d}{dz} g(z) \approx 0 \cdot (1-0) \approx 0$$

$$z = 0 \quad g(z) = \frac{1}{2}$$

$$\frac{d}{dz} g(z) = \frac{1}{2} \left( 1 - \frac{1}{2} \right) = \frac{1}{4}$$

# Tanh activation function



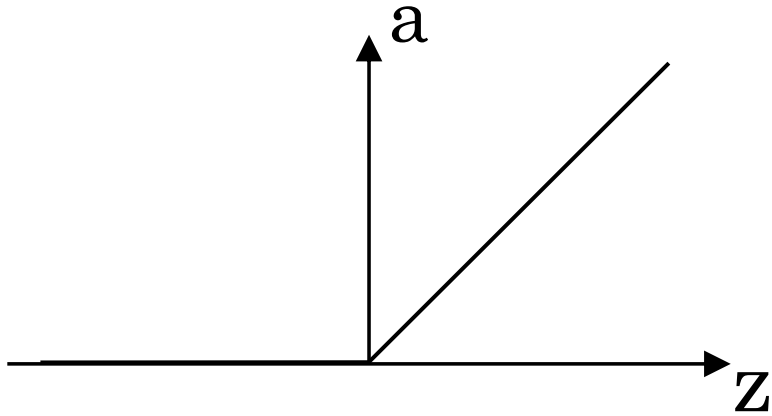
$$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = \frac{d}{dz} g(z) = \text{slope of } g(z) \text{ at } z = \underline{1 - (\tanh(z))^2} \leftarrow$$

$$a = g(z), \quad g'(z) = 1 - a^2$$

$$\left| \begin{array}{ll} z=10 & \tanh(z) \approx 1 \\ & g'(z) \approx 0 \\ z=-10 & \tanh(z) \approx -1 \\ & g'(z) \approx 0 \\ z=0 & \tanh(z) = 0 \\ & g'(z) = 1 \end{array} \right.$$

# ReLU and Leaky ReLU



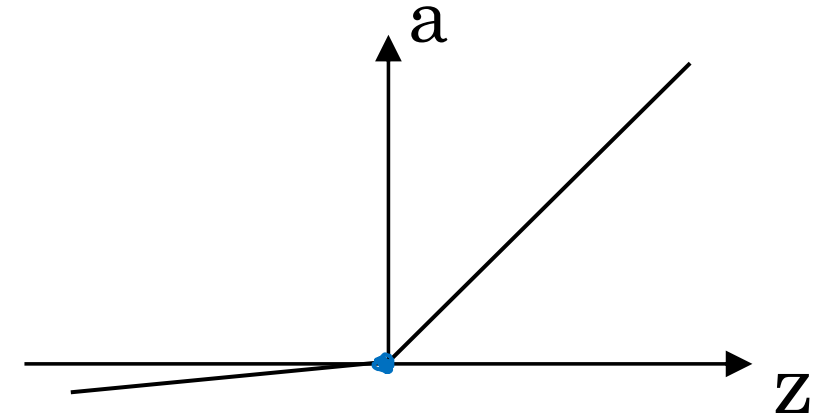
ReLU

$$g(z) = \max(0, z)$$

→  $g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$

~~undefined if  $z = 0$~~

$z = 0.0000 \dots 0$



# Leaky ReLU

$$g(z) = \max(0.01z, z)$$
$$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$



deeplearning.ai

One hidden layer  
Neural Network

---

Gradient descent for  
neural networks



# Gradient descent for neural networks

Parameters:  $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$   
 $(n^{[1]}, n^{[0]})$   $(n^{[1]}, 1)$   $(n^{[2]}, n^{[1]})$   $(n^{[2]}, 1)$

$$n_x = n^{[0]}, \quad n^{[1]}, \quad \underline{n^{[2]} = 1}$$

$$\text{Cost function: } J(W^{[1]}, b^{[1]}, \underline{W^{[2]}}, \underline{b^{[2]}}) = \frac{1}{m} \sum_{i=1}^m \ell(\hat{y}, y)$$

$\uparrow$   $\uparrow$   $\uparrow$   
 $a^{[2]}$

Gradient descent:

→ Repeat {

→ Compute predictions  $(\hat{y}^{(i)}, i=1, \dots, m)$

$$\underline{dW^{[1]}} = \frac{\partial J}{\partial W^{[1]}}, \quad \underline{db^{[1]}} = \frac{\partial J}{\partial b^{[1]}}, \quad \dots$$

$$W^{[1]} := W^{[1]} - \alpha dW^{[1]}$$

$$b^{[1]} := b^{[1]} - \alpha db^{[1]}$$

$$W^{[2]} := \dots \quad b^{[2]} := \dots$$

# Formulas for computing derivatives

Forward propagation:

$$z^{[1]} = w^{[1]}x + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]}) \leftarrow$$

$$z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]}) = \sigma(z^{[2]})$$

Back propagation:

$$dz^{[2]} = A^{[2]} - y \leftarrow$$

$$dw^{[2]} = \frac{1}{m} dz^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dz^{[2]}, \text{axis}=1, \text{keepdims}=\text{True})$$

$$dz^{[1]} = \underbrace{w^{[2]T} dz^{[2]}}_{(n^{[1]}, m)} \times \underbrace{g^{[1]'}(z^{[1]})}_{\text{element-wise product}} \quad (n^{[1]}, m)$$

$$dw^{[1]} = \frac{1}{m} dz^{[1]} x^T$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dz^{[1]}, \text{axis}=1, \text{keepdims}=\text{True})$$

$(n^{[1]}, 1)$   $(n^{[1]}, )$   $\uparrow$  reshape

$$Y = [y^{(1)} y^{(2)} \dots y^{(m)}]$$

$$(n^{[2]}) \leftarrow$$

$$\downarrow (n^{[2]}, 1) \leftarrow$$



deeplearning.ai

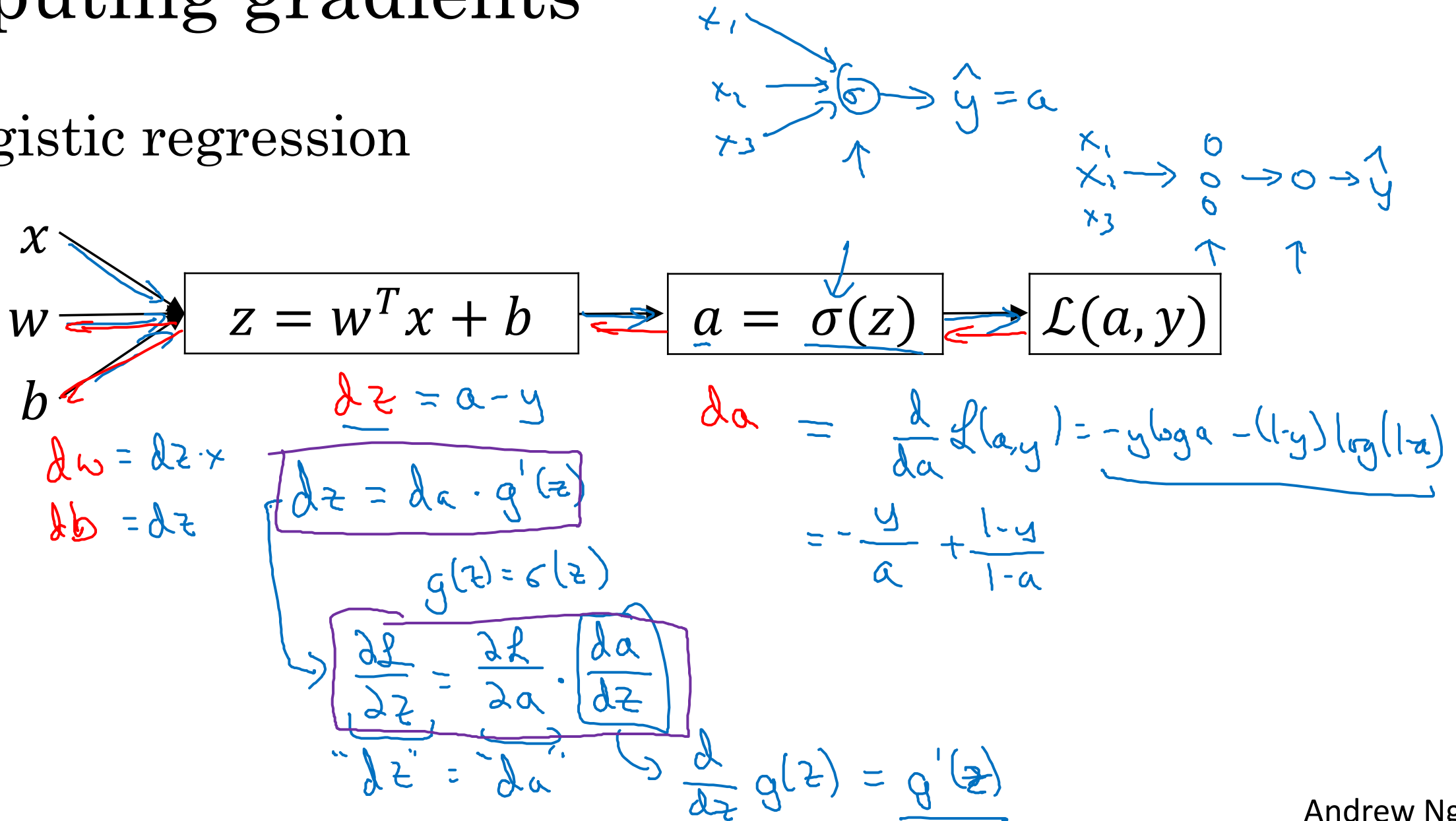
One hidden layer  
Neural Network

---

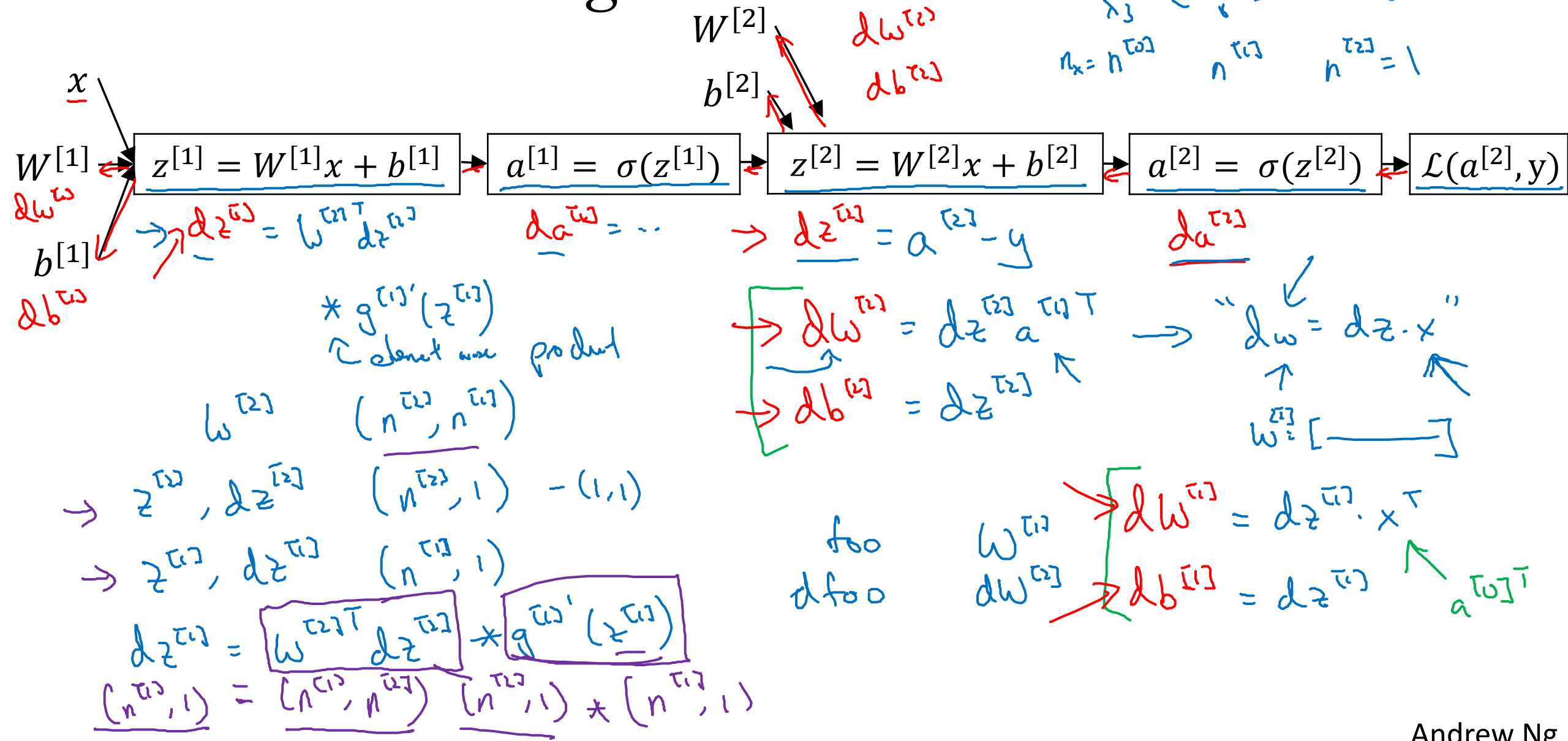
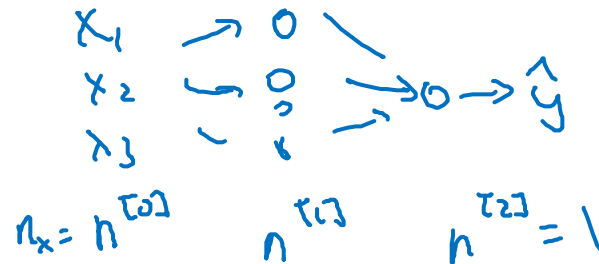
Backpropagation  
intuition (Optional)

# Computing gradients

## Logistic regression



# Neural network gradients



# Summary of gradient descent

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

Vectorized Implementation:

$$z^{[1]} = W^{[1]} x + b^{[1]}$$
$$a^{[1]} = g^{[1]}(z^{[1]})$$
$$z^{[1]} = \begin{bmatrix} z^{[1](1)} \\ z^{[1](2)} \\ \dots \\ z^{[1](n)} \end{bmatrix}$$
$$z^{[1]} = W^{[1]} X + b^{[1]}$$
$$A^{[1]} = g^{[1]}(z^{[1]})$$

# Summary of gradient descent

$$\underline{dz^{[2]}} = \underline{a^{[2]}} - \underline{y}$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$\underbrace{dz^{[1]}}_{(n^{[1]}, 1)} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

$$\underline{dZ^{[2]}} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$\underbrace{dZ^{[1]}}_{(n^{[1]}, m)} = \underbrace{W^{[2]T} dZ^{[2]}}_{(n^{[1]}, m)} * \underbrace{g^{[1]'}(Z^{[1]})}_{(n^{[1]}, m)}$$

↙ elementwise product

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$J(\cdot) = \frac{1}{m} \sum_{i=1}^n \mathcal{L}(\hat{y}_i, y_i)$$



deeplearning.ai

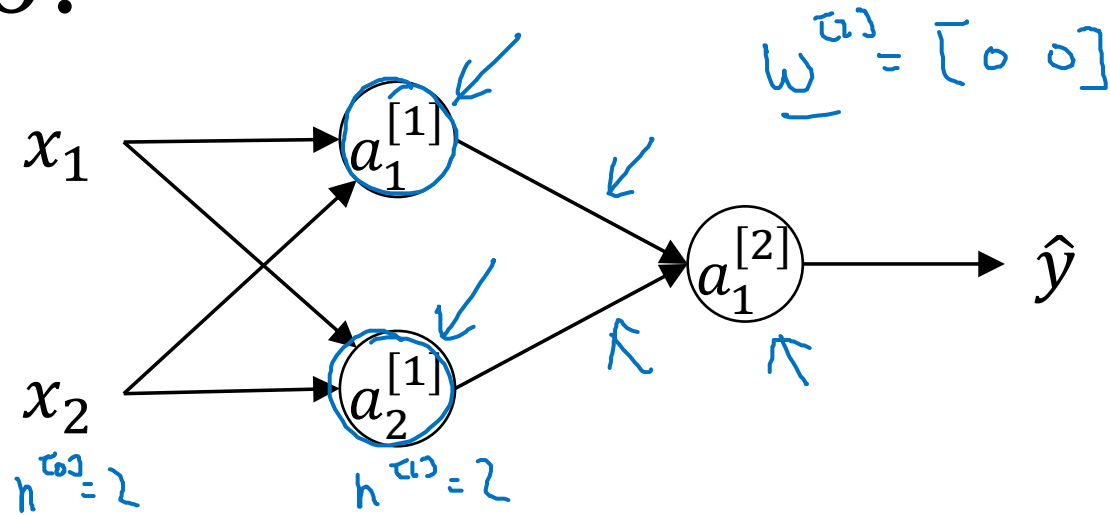
One hidden layer  
Neural Network

---

Random Initialization



# What happens if you initialize weights to zero?



$$\underline{w}^{(1)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$a_1^{(1)} = a_2^{(1)}$$

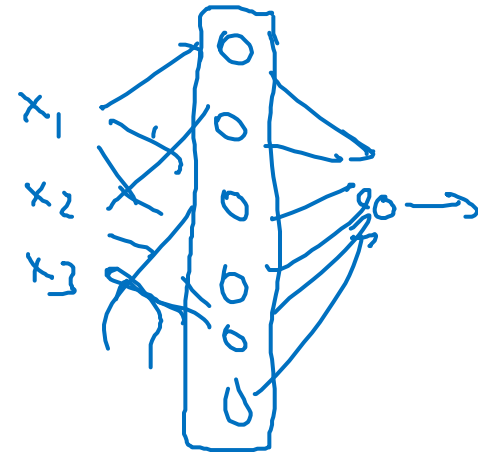
$$\underline{dw} = \begin{bmatrix} u & v \\ u & v \end{bmatrix}$$

$$\underline{b}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$dz_1^{(1)} = dz_2^{(1)}$$

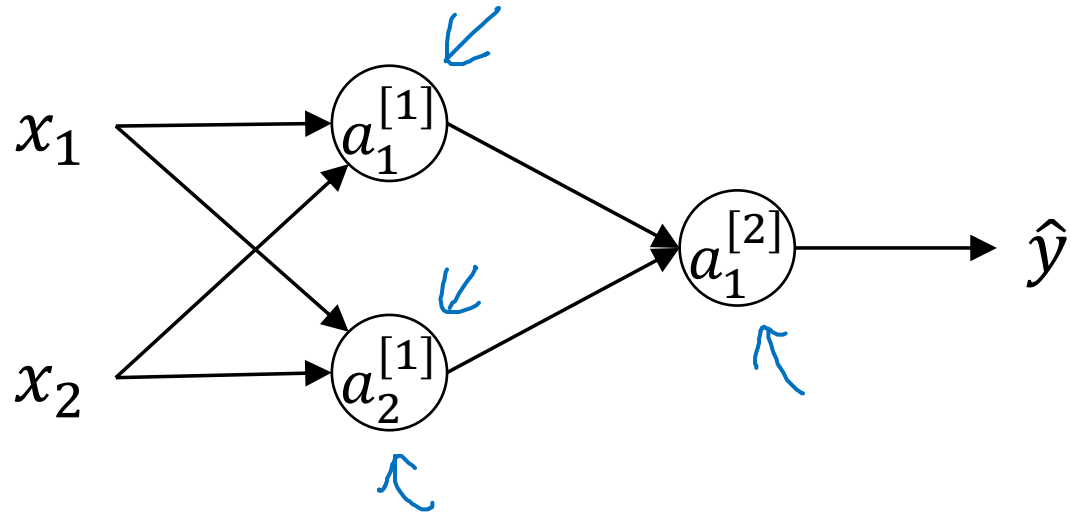
$$\underline{w}^{(1)} = \underline{w}^{(1)} - \underline{dw}$$

Symmetric

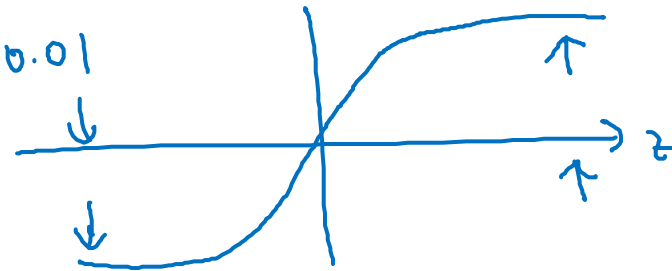


$$\underline{w}^{(1)} = \begin{bmatrix} \dots & \cdot \\ \dots & \cdot \end{bmatrix}$$

# Random initialization



→  $w^{[1]} = \text{np.random.randn}(2,2) * \frac{0.01}{100?}$   
 $b^{[1]} = \text{np.zeros}(2,1)$   
 $w^{[2]} = \text{np.random.randn}(1,2) * 0.01$   
 $b^{[2]} = 0$



$$z^{[1]} = w^{[1]}x + b^{[1]}$$
$$a^{[1]} = g^{[1]}(z^{[1]})$$



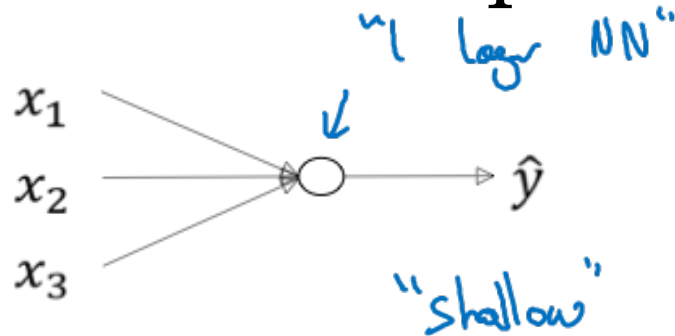
**deeplearning.ai**

# Deep Neural Networks

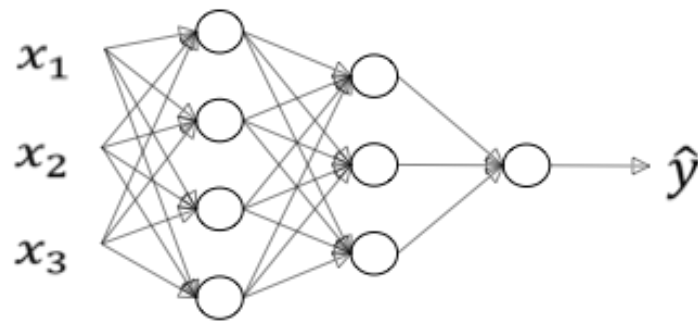
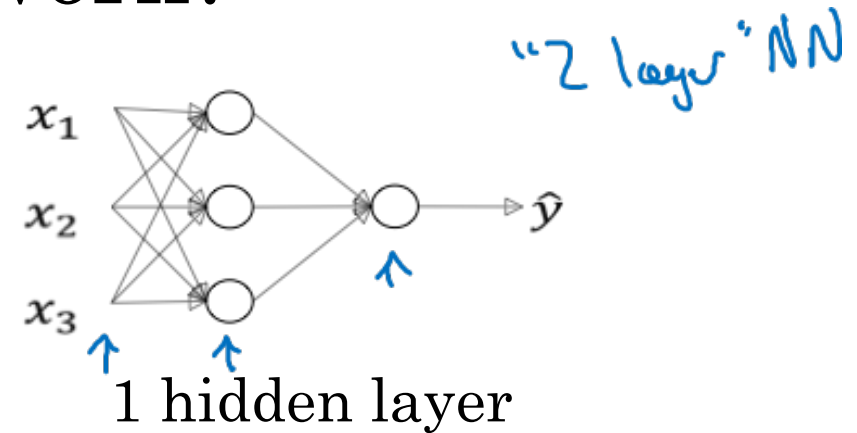
---

Deep L-layer  
Neural network

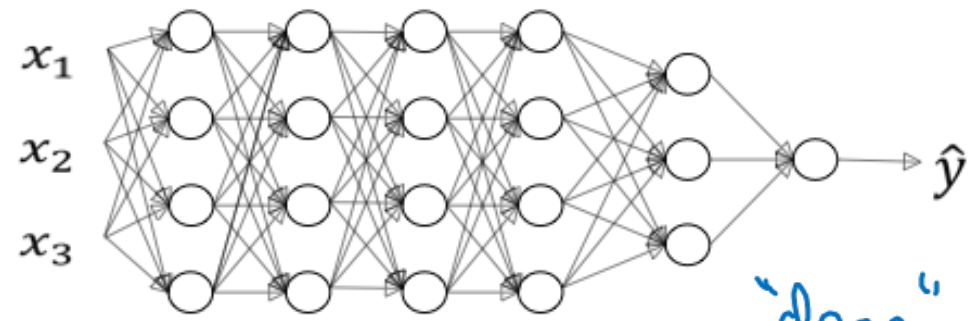
# What is a deep neural network?



logistic regression



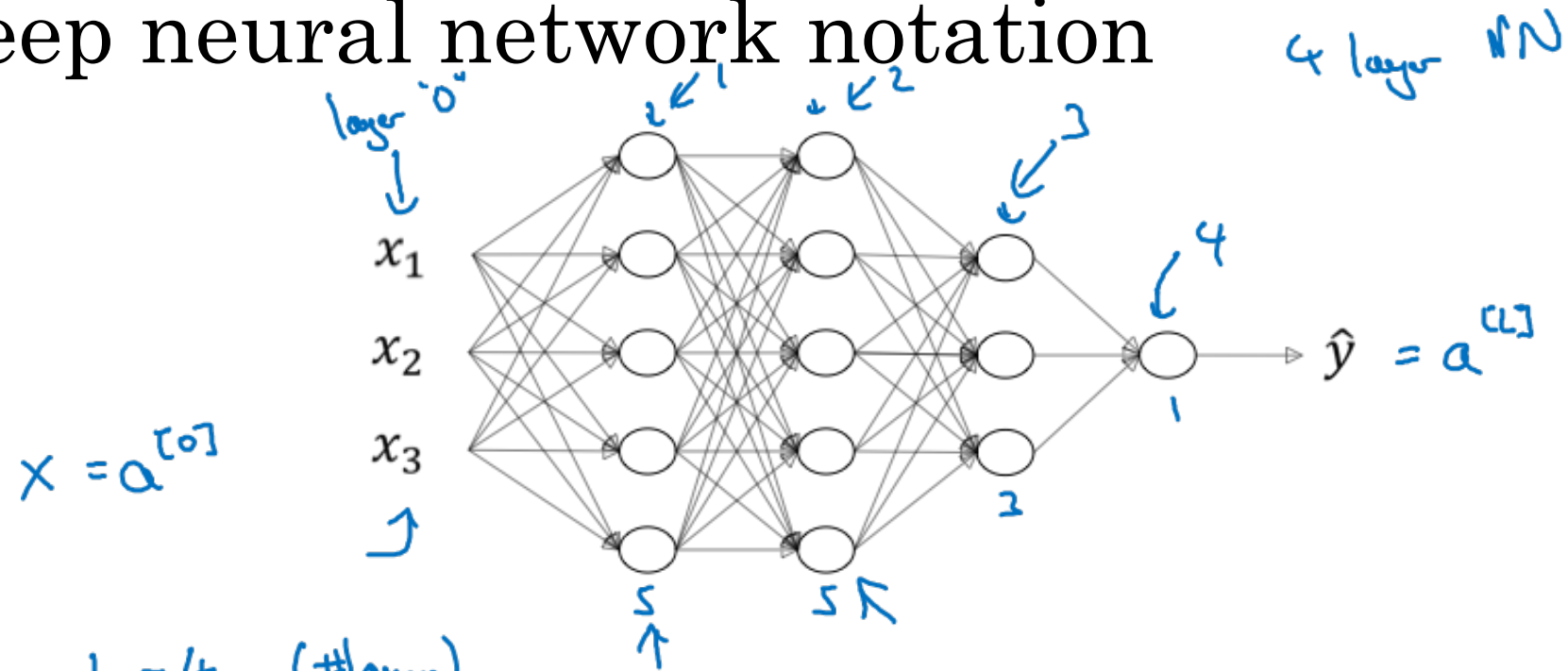
2 hidden layers



5 hidden layers

"deep"

# Deep neural network notation



$L = 4$  (#layers)

$n^{[l]} = \# \text{units in layer } l$

$a^{[l]} = \text{activations in layer } l$

$a^{[l]} = g(z^{[l]})$ ,  $w_{\delta a}^{[l]} = \text{weights for } \underline{z^{[l]}}$

$n^{[1]} = 5, n^{[2]} = 5, n^{[3]} = 3, n^{[4]} = n^{[L]} = 1$

$n^{[0]} = n_x = 3$



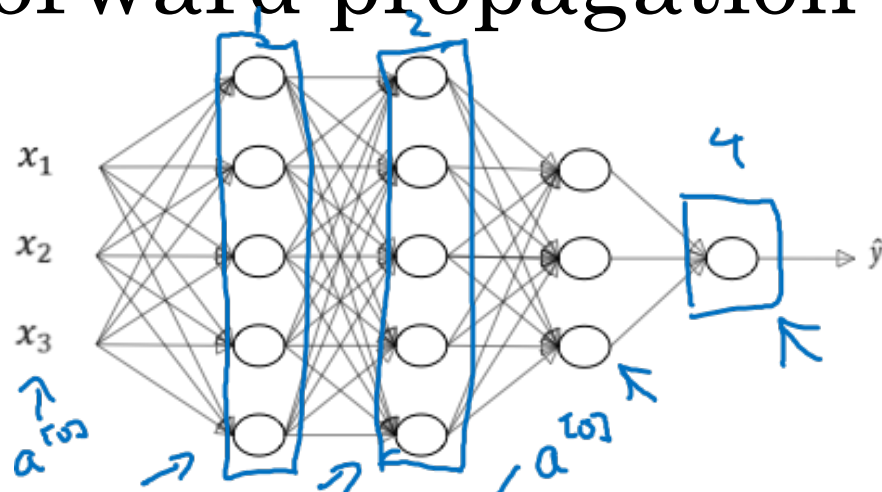
**deeplearning.ai**

# Deep Neural Networks

---

## Forward Propagation in a Deep Network

# Forward propagation in a deep network



$$\begin{aligned} z^{[l]} &= W^{[l]} A^{[l-1]} + b^{[l]} \\ A^{[l]} &= g^{[l]}(z^{[l]}) \end{aligned}$$

Handwritten note:  $A^{[0]} = X$

$$X : z^{[1]} = W^{[1]} a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]} a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$z^{[4]} = W^{[4]} a^{[3]} + b^{[4]}, a^{[4]} = g^{[4]}(z^{[4]}) = \hat{y}$$

$$\begin{bmatrix} z^{[1]}_1 & z^{[1]}_2 & \dots & z^{[1]}_n \\ z^{[2]}_1 & z^{[2]}_2 & \dots & z^{[2]}_n \\ \vdots & \vdots & \ddots & \vdots \\ z^{[4]}_1 & z^{[4]}_2 & \dots & z^{[4]}_n \end{bmatrix}$$

Vertical:

$$\begin{aligned} z^{[l]} &= W^{[l]} A^{[l-1]} + b^{[l]} \\ A^{[l]} &= g^{[l]}(z^{[l]}) \end{aligned}$$

for  $l=1 \dots 4$

Handwritten note:  $X = A^{[0]}$

Handwritten note:  $\hat{y} = g(z^{[4]}) = A^{[4]}$



deeplearning.ai

# Deep Neural Networks

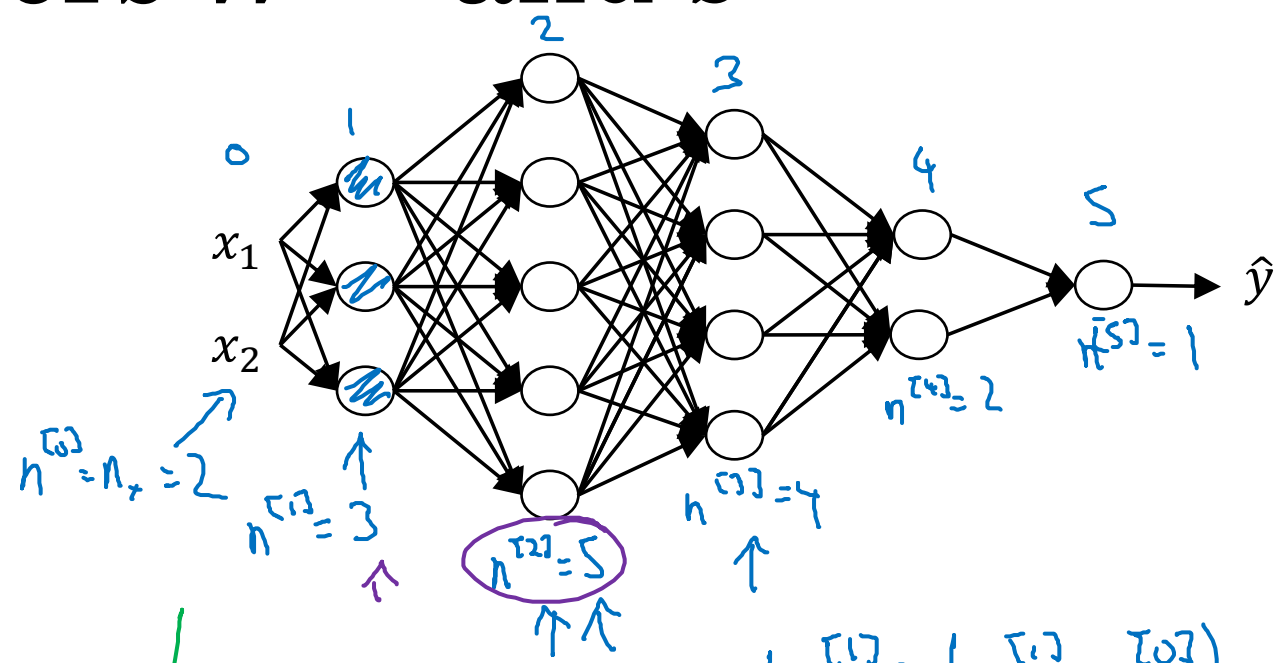
---

Getting your matrix  
dimensions right



# Parameters $W^{[l]}$ and $b^{[l]}$

$\downarrow$   
 $z^{[L]} = g^{[L]}(a^{[L]})$   
 $\uparrow$   
 $\downarrow$   
 $a^{[L]}$



$L=5$

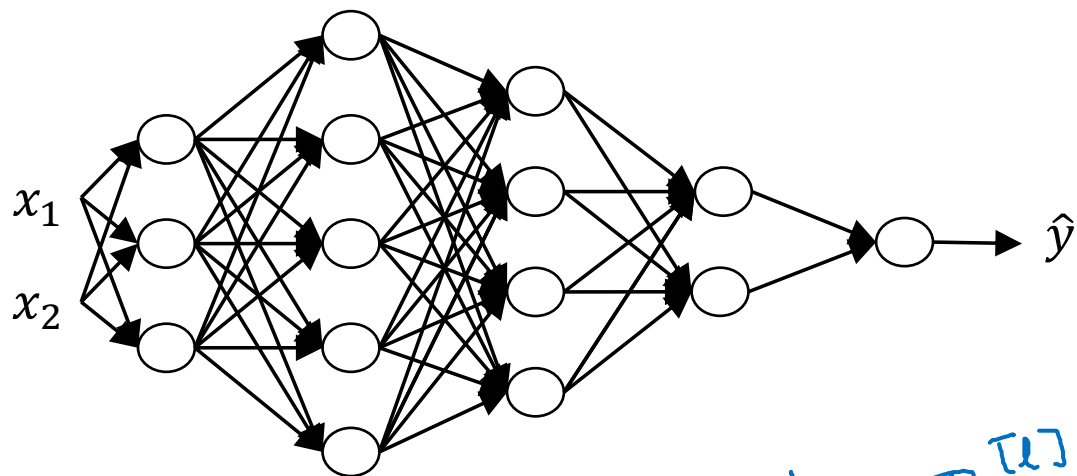
$\rightarrow W^{[L]}: (n^{[L]}, n^{[L-1]})$   
 $\rightarrow b^{[L]}: (n^{[L]}, 1)$   
 $\rightarrow \Delta W^{[L]}: (n^{[L]}, n^{[L-1]})$   
 $\rightarrow \Delta b^{[L]}: (n^{[L]}, 1)$

$\downarrow$   
 $z^{[1]} = \boxed{W^{[1]} \cdot x} + \boxed{b^{[1]}}$   
 $(3,1) \leftarrow (3,2) \quad (2,1)$   
 $(n^{[1]}, 1) \quad (n^{[1]}, n^{[0]}) \quad (n^{[0]}, 1)$   
 $(3,1)$   
 $(n^{[1]}, 1)$

$\begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix} = \begin{bmatrix} \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot \\ \cdot \end{bmatrix}$

$W^{[1]}: (n^{[1]}, n^{[0]})$   
 $W^{[2]}: (5, 3) \quad (n^{[2]}, n^{[1]})$   
 $z^{[2]} = \boxed{W^{[2]} \cdot a^{[1]}} + \boxed{b^{[2]}}$   
 $\uparrow \quad \uparrow \quad \uparrow$   
 $\rightarrow (5,1) \quad (5,3) \quad (3,1)$   
 $(5,1)$   
 $(n^{[2]}, 1)$   
 $W^{[3]}: (4, 5)$   
 $W^{[4]}: (2, 4)$ ,  $W^{[5]}: (1, 2)$

# Vectorized implementation



$$z^{[1]} = W^{[1]} \cdot x + b^{[1]}$$

$(n^{[1]}, 1)$      $(n^{[1]}, n^{[0]})$      $(n^{[0]}, 1)$      $(n^{[1]}, 1)$

$$[z^{[1](1)} \ z^{[1](2)} \ \dots \ z^{[1](m)}]$$

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

$(n^{[1]}, m)$      $(n^{[1]}, n^{[0]})$      $(n^{[0]}, m)$      $(n^{[1]}, 1)$   
 $\uparrow$      $\uparrow$      $(n^{[0]}, m)$      $(n^{[1]}, m)$

$$z^{[2]}, a^{[2]} : (n^{[2]}, 1)$$

$$z^{[2]}, A^{[2]} : (n^{[2]}, m)$$

$l=0 \quad A^{[0]} = X = (n^{[0]}, m)$

$$dz^{[2]}, dA^{[2]} : (n^{[2]}, m)$$



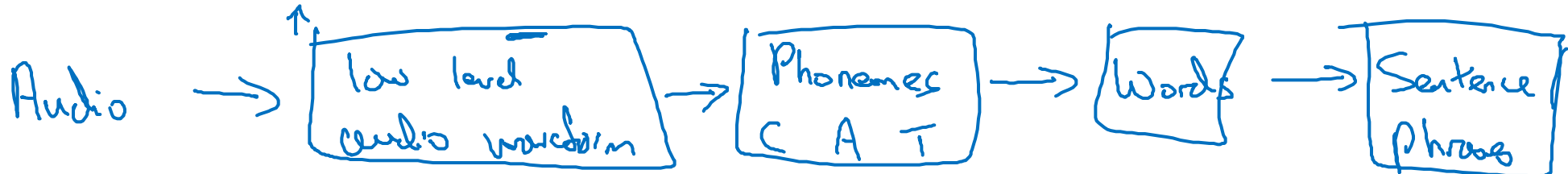
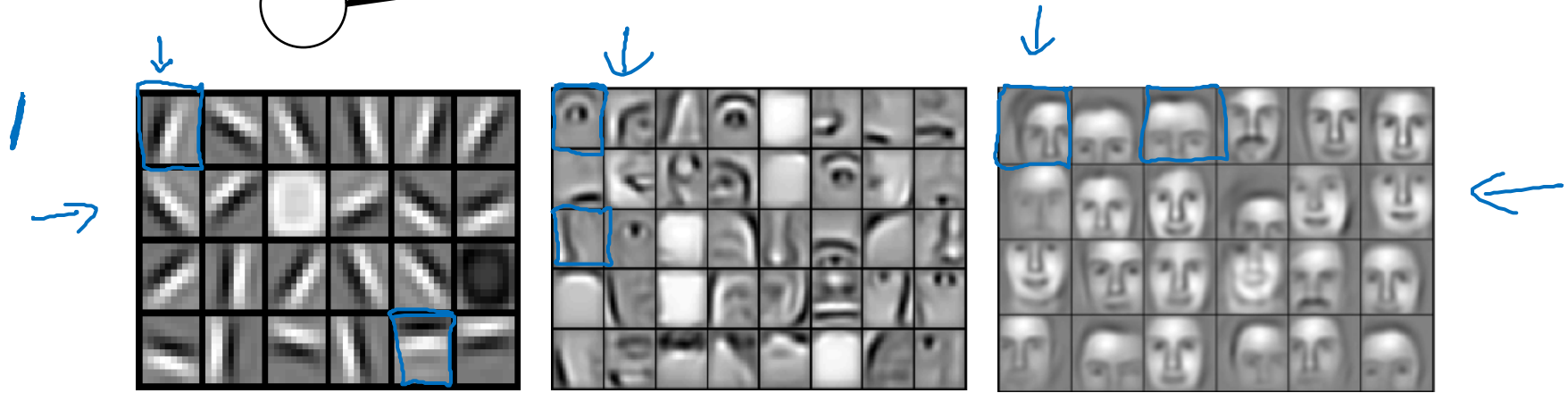
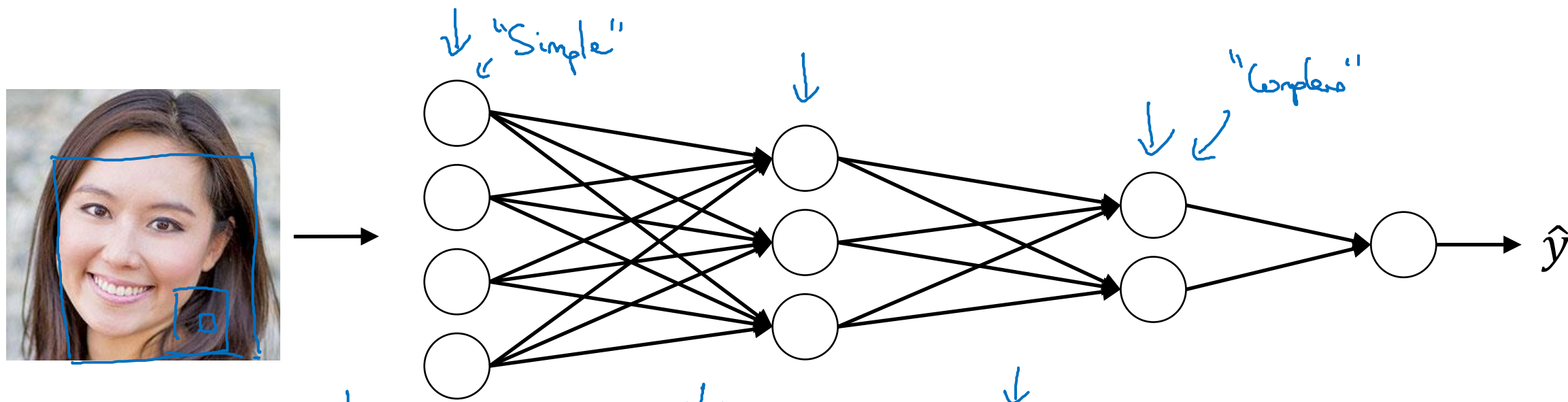
deeplearning.ai

# Deep Neural Networks

---

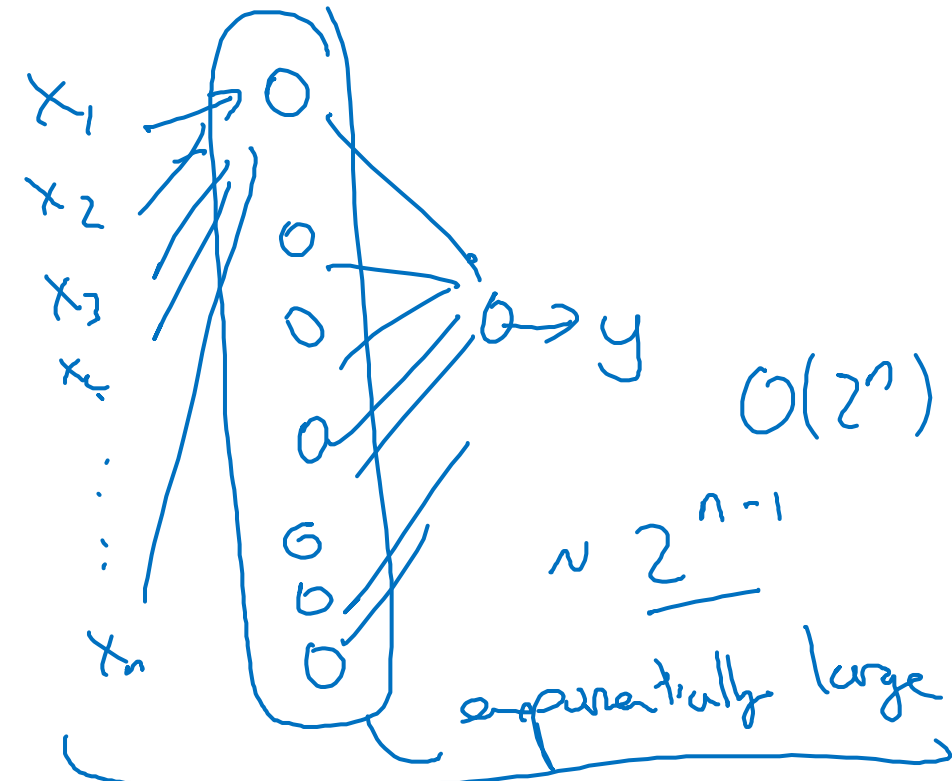
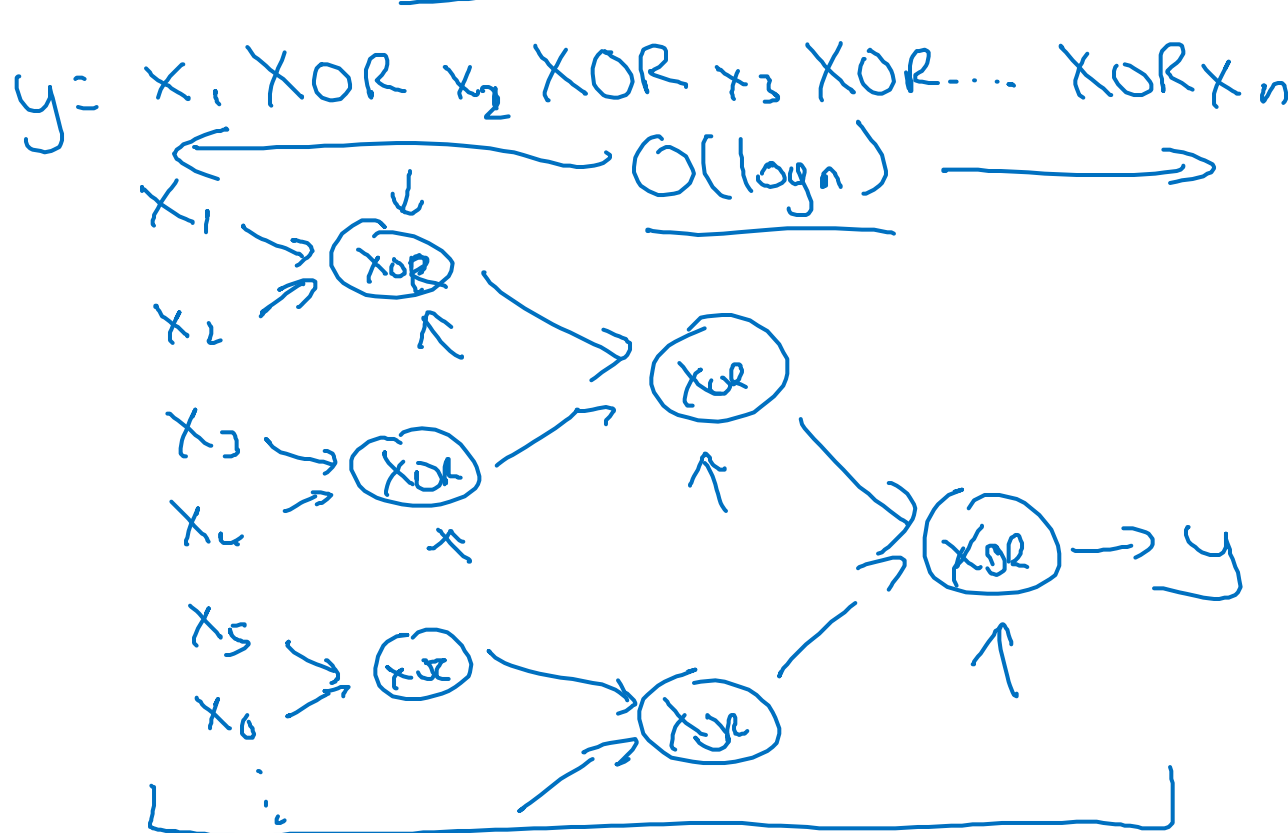
Why deep  
representations?

# Intuition about deep representation



# Circuit theory and deep learning

Informally: There are functions you can compute with a “small” L-layer deep neural network that shallower networks require exponentially more hidden units to compute.





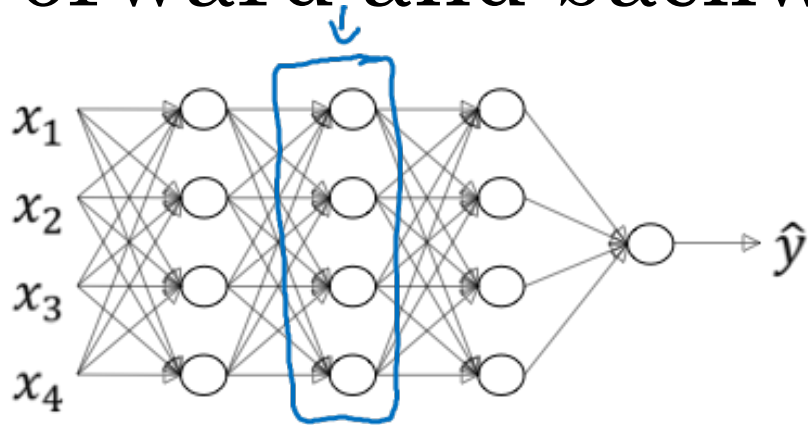
**deeplearning.ai**

# Deep Neural Networks

---

Building blocks of  
deep neural networks

# Forward and backward functions



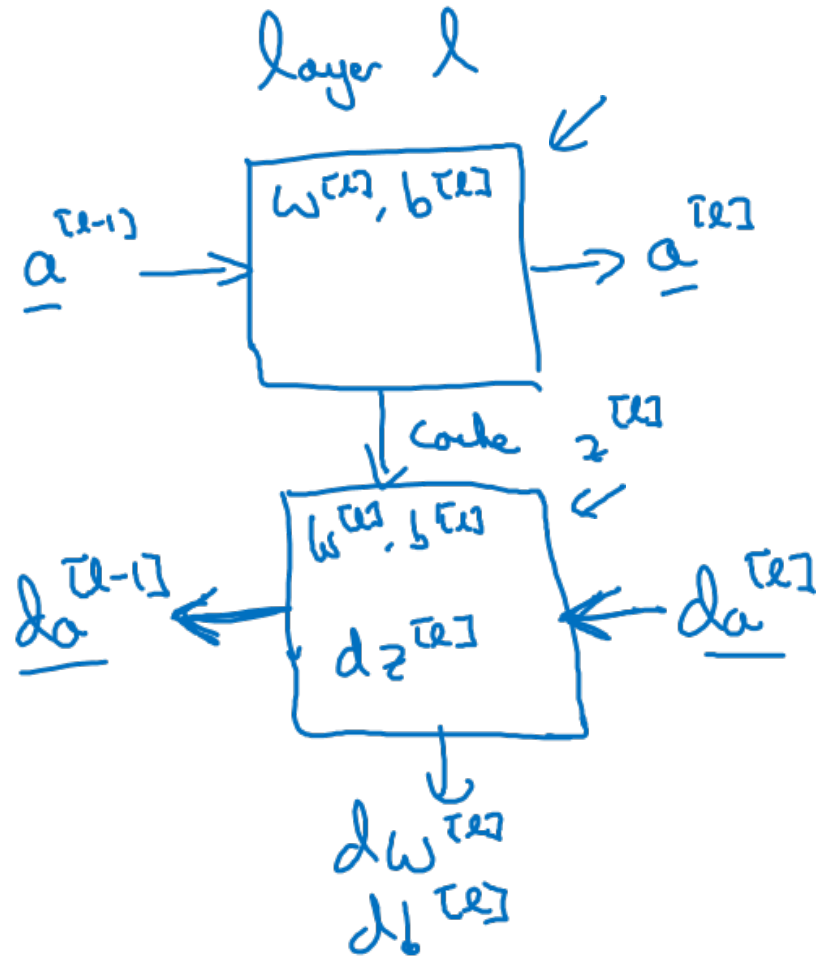
layer  $l$ :  $W^{[l]}, b^{[l]}$

→ Forward: Input  $a^{[l-1]}$ , output  $a^{[l]}$

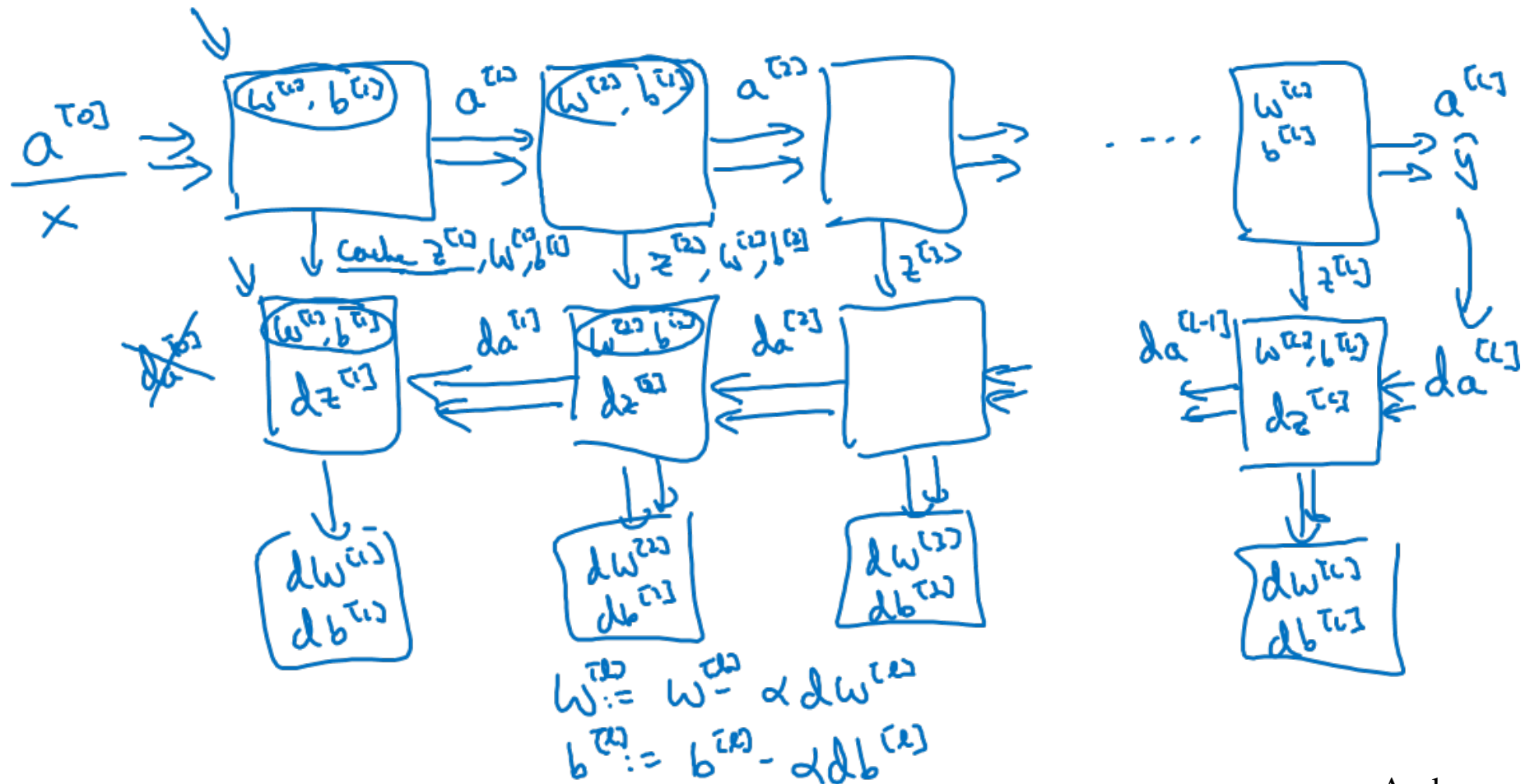
$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]} \quad \text{cache } z^{[l]}$$

$$\underline{a}^{[l]} = g^{[l]}(z^{[l]})$$

→ Backward: Input  $da^{[l]}$ , output  $da^{[l-1]}$   
 cache  $(z^{[l]})$   
 $\frac{dW^{[l]}}{db^{[l]}}$



# Forward and backward functions







deeplearning.ai

# Deep Neural Networks

---

Forward and backward  
propagation

# Backward propagation for layer $l$

→ Input  $da^{[l]}$

→ Output  $da^{[l-1]}, dW^{[l]}, db^{[l]}$

$$dz^{[l]} = da^{[l]} * g^{[l]'}(z^{[l]})$$

$$dW^{[l]} = dz^{[l]} \cdot \underline{a^{[l-1]}}$$

$$db^{[l]} = dz^{[l]}$$

$$da^{[l-1]} = W^{[l]T} \cdot dz^{[l]}$$

$$dz^{[l]} = W^{[l+1]T} dz^{[l+1]} * g^{[l]'}(z^{[l]})$$

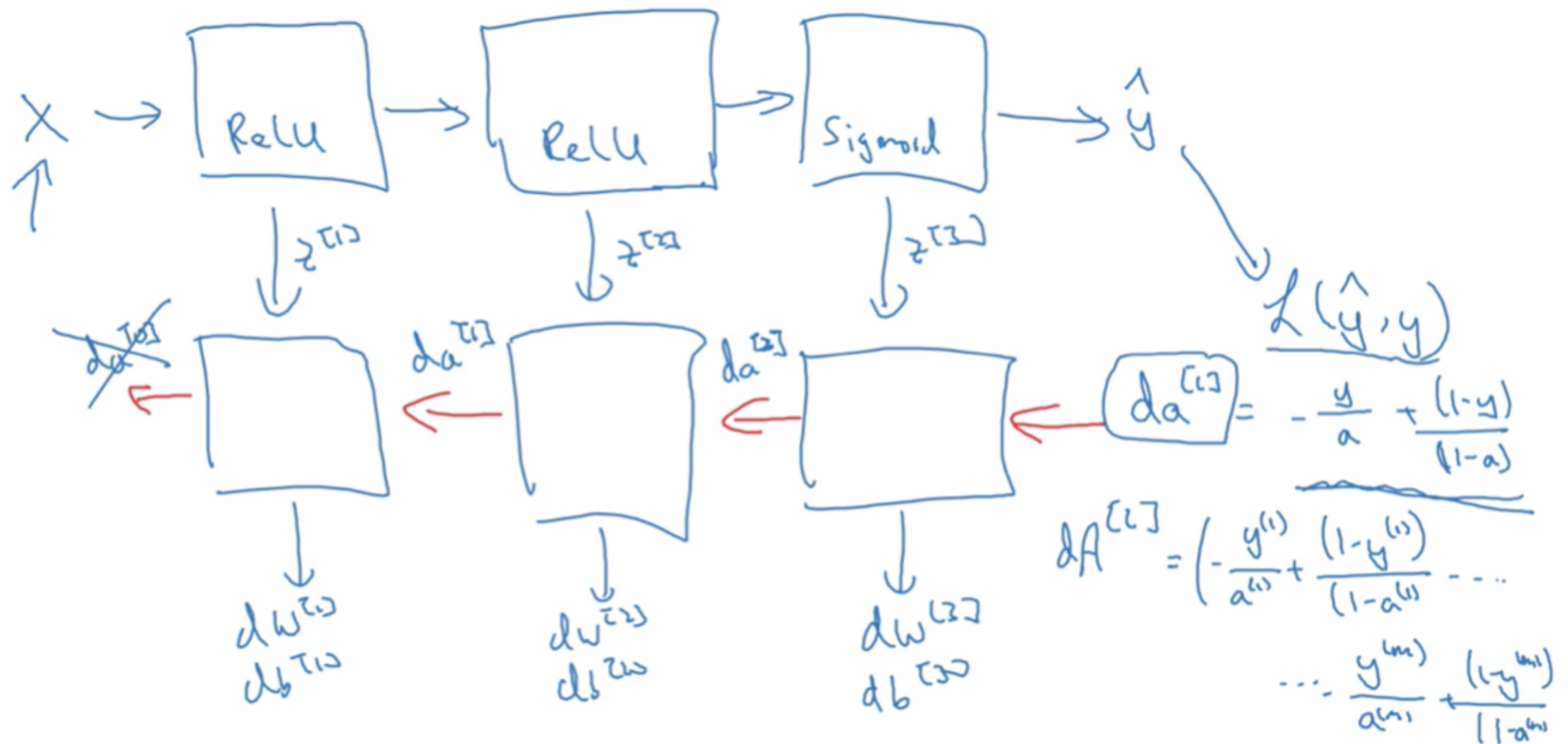
$$dz^{[l]} = dA^{[l]} * g^{[l]'}(z^{[l]})$$

$$dW^{[l]} = \frac{1}{n} dz^{[l]} \cdot A^{[l-1]T}$$

$$db^{[l]} = \frac{1}{n} np.sum(dz^{[l]}, axis=1, keepdims=True)$$

$$dA^{[l-1]} = W^{[l]T} \cdot dz^{[l]}$$

# Summary





deeplearning.ai

# Deep Neural Networks

---

## Parameters vs Hyperparameters

# What are hyperparameters?

Parameters:  $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, b^{[3]} \dots$

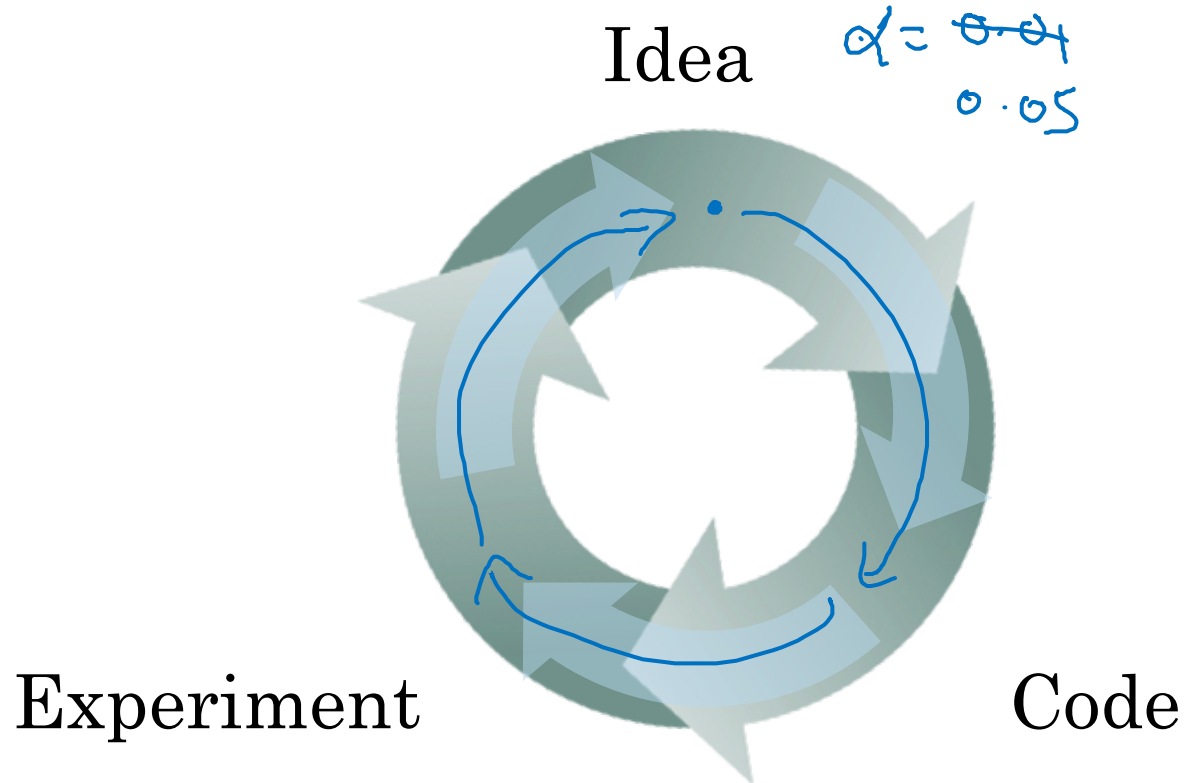


Hyperparameters:

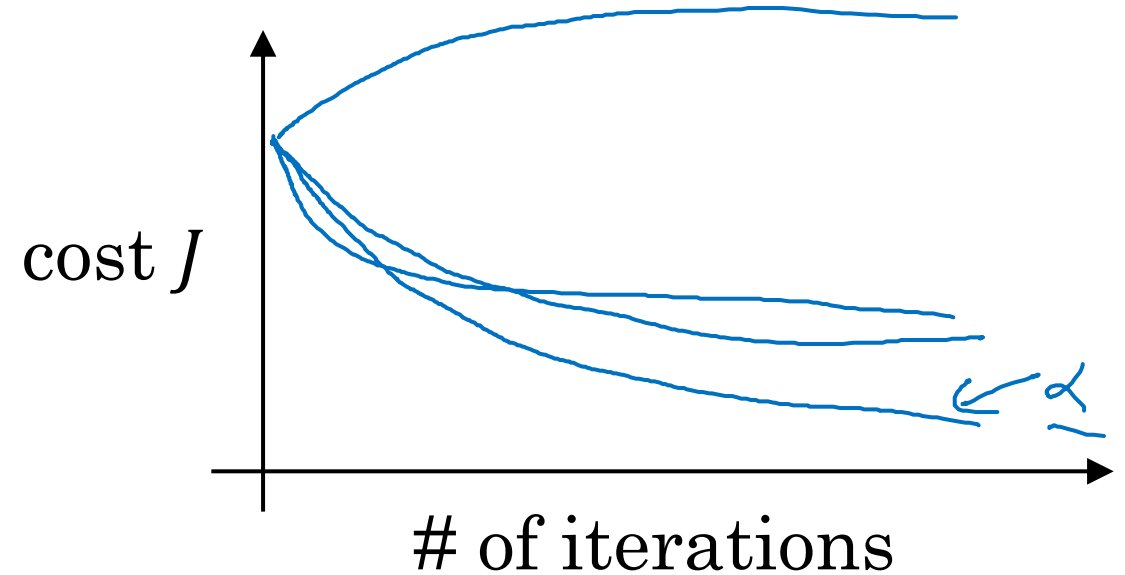
- learning rate  $\alpha$
- $\tau$
- #iterations
- #hidden layers  $L$
- # hidden units  $n^{[1]}, n^{[2]}, \dots$
- choice of activation function

Later: Momentum, mini-batch size, regularizations, ...

# Applied deep learning is a very empirical process



Vision, Speech, NLP, Ad, Search, Recommendation.





deeplearning.ai

# Deep Neural Networks

---

What does this  
have to do with  
the brain?

# Forward and backward propagation

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = g^{[1]}(Z^{[1]})$$

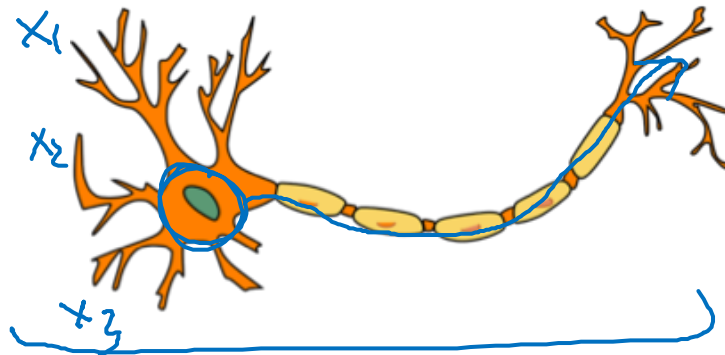
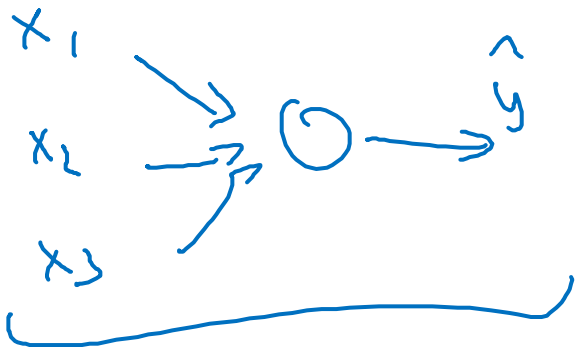
$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(Z^{[2]})$$

$$\vdots$$

$$A^{[L]} = g^{[L]}(Z^{[L]}) = \hat{Y}$$

"It's like the brain"



$$dZ^{[L]} = A^{[L]} - Y$$

$$dW^{[L]} = \frac{1}{m} dZ^{[L]} A^{[L]T}$$

$$db^{[L]} = \frac{1}{m} \text{np.sum}(dZ^{[L]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$dZ^{[L-1]} = dW^{[L]T} dZ^{[L]} g'^{[L]}(Z^{[L-1]})$$

$$\vdots$$
$$dZ^{[1]} = dW^{[L]T} dZ^{[2]} g'^{[1]}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} A^{[1]T}$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True})$$